分类号	
UDC	
密　级	
学　号	

西安理工大学

硕士学位论文

中国象棋计算机博弈数据结构与评估函数的研究和实现

谢艳茹

学 科 名 称：控制理论与控制工程

学 科 门 类：工　学

指 导 教 师：钱富才　教　授

申 请 日 期：2008 年 3 月

中国知网 https://www.cnki.net

独 创 性 声 明

秉承祖国优良道德传统和学校的严谨学风郑重申明：本人所呈交的学位论文是我个人在导师指导下进行的研究工作及取得的成果。尽我所知，除特别加以标注和致谢的地方外，论文中不包含其他人的研究成果。与我一同工作的同志对本文所研究的工作和成果的任何贡献均已在论文中作了明确的说明并已致谢。

本论文及其相关资料若有不实之处，由本人承担一切相关责任

论文作者签名：谢艳荔　　2008年 3 月 27 日

学位论文使用授权声明

本人谢艳荔在导师的指导下创作完成毕业论文。本人已通过论文的答辩，并已经在西安理工大学申请博士 / 硕士学位。本人作为学位论文著作权拥有者，同意授权西安理工大学拥有学位论文的部分使用权，即：1）已获学位的研究生按学校规定提交印刷版和电子版学位论文，学校可以采用影印、缩印或其他复制手段保存研究生上交的学位论文，可以将学位论文的全部或部分内容编入有关数据库进行检索；2）为教学和科研目的，学校可以将公开的学位论文或解密后的学位论文作为资料在图书馆、资料室等场所或在校园网上供校内师生阅读、浏览。

本人学位论文全部或部分内容的公布（包括刊登）授权西安理工大学研究生部办理。

（保密的学位论文在解密后，适用本授权说明）

论文作者签名：谢艳荔　导师签名：钱富才　　2008年 3 月 27 日

中国知网　https://www.cnki.net

论文题目：中国象棋计算机博弈数据结构与评估函数的研究和实现

学科专业：控制理论与控制工程

研 究 生：谢艳茹　　签 名：谢艳茹

指导导师：钱富才 教授　　签 名：钱富才

摘 要

计算机博弈被认为是人工智能领域最具有挑战性的研究方向之一。人工智能的先驱们曾表明，如果掌握了下棋的本质，也许就掌握了人类智能行为的核心，计算机博弈为人工智能提供了一个良好的试验场所。国际象棋的计算机博弈已经有了很长的历史，并且取得了巨大的成绩，而历史悠久的中国象棋的计算机博弈才刚刚起步，很多技术还不够成熟。随着对人工智能技术的深入研究，越来越多的学者把目光放在了中国象棋博弈的研究上，并把其看作是相关研究的重点之一。

本文在大量阅读国际象棋成熟技术的基础上，结合中国象棋计算机博弈近年来的发展实践，对计算机博弈核心技术中的数据结构和评估函数进行了深入地研究与探讨。数据结构是计算机博弈实现的基础，构架一个好的数据结构对整个引擎的速度影响巨大，对后续博弈技术的实现有着决定性的作用。而除了搜索之外，评估函数是计算机博弈中最重要的部分，也是最人性化的一部分，它在很大程度上决定了博弈水平的高低。鉴于此，本文的主要研究内容如下：

(1) 分析总结了计算机博弈的关键技术要点，包括数据结构、博弈树搜索技术、评估函数等。

(2) 研究并设计了适合中国象棋计算机博弈的棋局表示方法和着法生成方法，即数据结构。解决了如何把棋盘分布信息和棋盘状态信息编码进计算机的问题，并使用着法生成辅助数据结构加速了着法生成。

(3) 深入研究了评估函数的构成，对评估函数建立数学模型，分析研究了一些可应用于估值优化的方法和策略。在设计系统的评估函数时，主要采用手工调整方法进行估值参数优化，并在评估函数中使用藐视因子和局势因子来调整着法路径的选择，有效地提高了对弈获胜率。

(4) 系统的开发和实现。

关键词：计算机博弈；中国象棋；数据结构；评估函数；估值优化

中国知网 https://www.cnki.net

Title: RESEARCH AND IMPLEMENTATION OF DATA STRUCTURE AND EVALUATION IN COMPUTER CHINESE CHESS

Major: Control Theory and Control Engineering

Name: Yanru XIE **Signature:** Xie Yanru

Supervisor: Prof. Fucai QIAN **Signature:** Qian Fucai

Abstract

Computer game is one of the most challenging topics in the field of artificial intelligence. The trailblazers in artificial intelligence once said: if mastered the essence of chess, maybe master the core of human intelligent behavior. Computer game offers a well testing ground to artificial intelligence. Chess computer game has a long history, and has acquired great success. Whereas, Chinese chess computer game starts just now and many techniques haven't matured enough though Chinese chess is an old-line game. With deeper studying on artificial intelligence, more and more scholars put their view on Chinese chess computer game, and regard it as one of the correlative research emphases.

Based on lots of techniques of chess computer game, years development and practice of Chinese chess computer game, the paper discusses and researches deep on data structure and evaluation function. Data structure is the foundation of designing computer game. An effective data structure has great influence on the whole engine, and it is the basic to achieve other technics. Moreover, besides search engine, evaluation is most important, and also is a part which most embodies the human nature. It determines the level of games in the most situations. Wherefore, the main points are as follow:

(1) Analyse and summarize the technical cruces about computer game, such as data structure, searching game tree, evaluation function, and so on.

(2) Study and design the methods of chessboard denotation and move generator which are suitable for Chinese chess computer game, that is data structures. Solve how encode the board distribution information and board state information into computer, and use assistant data structures to speed up move generator.

(3) Research deep on composing of evaluation, and establish mathematic model to it. Analyse and research some methods and strategies which can be used to optimize parameters. Use handwork methods to design evaluation and optimize parameters mainly in my system,

中国知网 https://www.cnki.net

and add contempt factor and situational factor to evaluation to adjust the pathes of moving which enhance the rates of victory in the game.

(4) Introduce system design and realization.

Key words: computer game; Chinese chess; data structure; evaluation function; optimize parameters

中国知网 https://www.cnki.net

目 录

1 绪论 1
1.1 研究背景 1
1.1.1 中国象棋计算机博弈的发展历程 1
1.1.2 象棋博弈过程的建模 3
1.2 数据结构和评估函数在中国象棋博弈中的作用 4
1.3 本文的主要工作 5
2 数据结构 7
2.1 局面表示 7
2.1.1 棋盘表示 7
2.1.2 棋子表示 9
2.1.3 棋盘分布信息表示 10
2.1.4 棋盘状态信息表示 13
2.2 着法生成 16
2.2.1 模板匹配法 17
2.2.2 预置着法表 18
2.2.3 测试实验研究 19
2.3 小结 20
3 博弈树展开与搜索 21
3.1 博弈树的展开 21
3.2 博弈树搜索 22
3.2.1 极大极小值搜索算法 22
3.2.2 a-ß 剪枝搜索 23
3.2.3 其他搜索算法 25
3.3 小结 25
4 评估函数 27
4.1 评估函数与博弈性能 27
4.2 评估函数建模 28
4.2.1 固定子力值的评价 28
4.2.2 棋子位置值的评价 28
4.2.3 棋子灵活度的评价 31
4.2.4 棋子的协调性、威胁与保护 32
4.2.5 特殊棋型的评价 34
4.2.6 对战术的评价 34
4.2.7 评估函数模型 34

中国知网 https://www.cnki.net

4.3 估值参数优化 35
4.3.1 手工调整 36
4.3.2 机器自学习 36
4.4 参数优化策略 39
4.4.1 藐视因子 39
4.4.2 局势因子 40
4.5 全动态调整估值参数 44
4.6 评估函数优劣的测试 46
4.7 小结 46
5 博弈系统开发与实现 48
5.1 系统程序的主要组件 48
5.1.1 数据结构 49
5.1.2 评估函数 52
5.1.3 用户界面 53
5.2 测试平台 54
6 总结及展望 55
致 谢 56
参考文献 57

中国知网 https://www.cnki.net

1 绪论

1.1 研究背景

博弈(Game Playing)是指一些个人、团体、或其他组织，面对一定的环境条件，在一定的约束条件下，依靠所掌握的信息，同时或先后，一次或多次，从各种可能的行为或策略集合中进行选择并实施，各自从中取得相应结果或收益的过程。可见博弈是一种竞争，竞争现象广泛存在于社会活动的许多方面，博弈论(Game Theory)在经济、社会、政治、军事等众多领域都已经得到广泛关注和应用[1-2]。

早在人类文明发展的初期，人类便开始进行棋类博弈的游戏了，而象棋又是各类棋类游戏中最受欢迎的一种智力游戏。象棋的计算机博弈就是计算机下象棋。1950 年，信息论的创始人香农(Claude E. Shannon)教授提出了象棋博弈编程的方案，成为计算机博弈的创始人[3]。

1956 年人工智能学科在美国达特茅斯大学的国际会议上诞生，经过半个世纪的发展，人工智能的作用和影响已经在各个领域发挥的淋漓尽致，对现实社会做出了非常重大的贡献，尤其是在自动化和工业控制领域，人工智能的应用更加突出[4-5]。而计算机博弈在人工智能研究中占有重要的地位，被认为是人工智能领域最具挑战性的研究方向之一，许多学者认为，对于人工智能研究而言，象棋的重要作用相当于遗传学研究中的果蝇。

象棋是一种完全知识博弈(Games of Perfect Information)，意思是指参与博弈的双方在任何时候都完全清楚每一个棋子是否存在，位于何处。象棋博弈具有下列基本特征[3]：

(1) 规则简单明确，胜负判断的标准简单，不包含任何机会或偶然性；

(2) 问题的状态（即棋盘上的局面）数量在数学意义上是有限的；

(3) 问题的解决在认识意义上不需要过多的知识。

上述特点吸引了许多领域的学者将象棋的计算机博弈作为研究对象。

1.1.1 中国象棋计算机博弈的发展历程

中国象棋计算机博弈技术是在国际象棋计算机博弈的基础上发展起来的，主要经历了四个阶段[6-8]。

第一个阶段是起源阶段。早在计算机发明之前，人们就设想如何让机器下棋。1769 年，匈牙利工程师巴朗·沃尔夫冈·凡·坎比林(Baron Wolfgang von Kempelen)为奥地利皇后做了一台会下国际象棋的机器，这是一个外形呆板的机械装置，它的出色棋力来自于有一名象棋高手巧妙地藏在机器里，所以这台“机器”只是个冒牌货；第一个棋弈程序写于电脑被真正发明之前，阿伦·图灵(Alan Turing)根据想象力写下了能够让机器下棋的指令，他知道可编程电脑即将出现，一旦发明出来，就有下棋的能力。由于当时还没有一台机器能够执行这些指令，于是他就自己执行，充当一个“人类 CPU”，每走一步需要半个多小时，称为“纸上机器”。

中国知网 https://www.cnki.net

第二个阶段是探索阶段。香农发表在荷兰著名杂志《哲学》上的一篇文章 *Programming a computer for playing chess* 奠定了现代计算机象棋的基础，香农(1950)和图灵(1953)提出了对棋类博弈程序的描述；1946 年，美籍匈牙利数学冯·诺依曼(John von Neumann)被指派设计一台强大的计算机器以加快工作进度，1950 年一台叫“MANIAC 一号”的巨型机被交付使用，它内装有数千个电子管和开关，每秒能执行 10,000 条指令，为了试验一下这台机器，科学家首先编写了一个下棋的程序，这是一个缩小的6×6棋盘，没有象，虽然作了简化但程序要搜索四层的深度仍需要 12 分钟。

第三个阶段是国际象棋计算机博弈快速发展阶段。1958 年，IBM704 成为第一台能同人下棋的计算机，名为“思考”；1973 年，CHESS 4.0 被 B.Slate 和 Atkin 开发出来，成为未来程序的基础；从 1980 年开始，电脑博弈开始逐渐向人类智能发起了挑战，电脑科学家肯·汤普森(Ken Thompson)和贝尔实验室的同事一起，使用了价值大约 20,000 美元的几百个芯片，建造了一台名为“尤物”(belle)的机器，它能够每秒搜索大约 18 万个局面，八至九层那么深，从 1980 年到 1983 年它赢得了世界电脑国际象棋和所有其它电脑竞赛冠军；80 年代中期，美国卡梅隆大学的汉斯·贝利纳(Hans Berliner)教授接手肯·汤普森放下的工作，制造了一台硬件型的机器叫“高技术”(HiTech)，随后贝利纳的几个学生包括华人许锋雄等自行研究叫“芯测”的机器，后来则是“深思”(Deep Thought)，1989 年，“深思”已经有 6 台信息处理器，每秒思考速度达 200 万步，成为第一台国际象棋大师级的计算机；许锋雄等后来加入了 IBM，和其他人合作制造了 IBM 现在的“深蓝”(Deep Blue)，1997 年，“深蓝”以 3.5 比 2.5 战胜卡斯帕罗夫，“深蓝”的胜利称作是人工智能领域的一个重要里程碑。

第四个阶段是中国象棋计算机博弈发展阶段。在国际象棋计算机博弈研究取得了巨大进步的时候，具有悠久历史的中国象棋的计算机博弈开展的并不顺利，缺少学者的关注，参考文献寥寥无几，使中国象棋计算机博弈面临艰难的局面。台湾学者在七十年代末开始了中国象棋计算机博弈的研究工作，1983 年南开大学开发了残局程序，之后开发了全局程序。近年来，通过许多中国象棋软件编程爱好者和多个象棋开发团队的努力，中国象棋软件水平有了长足的进步，每年也会有全世界范围内的中国象棋计算机博弈软件之间的比赛。虽然如此，由于中国象棋在计算机实现方面比国际象棋更加复杂，中国象棋计算机博弈的研究比国际象棋还落后很多，水平距离世界冠军差距甚远，即使最好的软件也仅仅接近大师的水平，而从事这一领域研究工作的学者更是少之又少，很多方面有待深入地研究。

国际象棋棋盘 8 行 8 列共 64 格，中国象棋是 10 行 9 列共 90 个交点，这表明中国象棋的运子空间更大。表 1-1 是国际象棋(Chess)，中国象棋(Chinese Chess)，日本将棋(Shogi)以及围棋(Go)的空间复杂度和博弈树复杂度的比较，其中表中的数字为复杂度的自然对数值[9, 16]。从表中可以看出，中国象棋的空间复杂度和国际象棋差不多，但树的复杂度比国际象棋要高。

中国知网 https://www.cnki.net

表 1-1 几种棋类的空间复杂度及树的复杂度

Table 1-1 State-space complexity and game tree complexity of some chesses

	空间复杂度	树的复杂度
Chess	50	123
Chinese chess	48	150
Shogi	71	226
Go	160	400

1.1.2 象棋博弈过程的建模

一个完整的中国象棋计算机博弈程序是一个系统工程，主要包括四个部分，即数据结构，博弈树搜索技术，评估函数，其他（开局库、残局库等）。为了深入研究计算机博弈的原理，尤其是从系统论和博弈论的角度研究问题的规律和求解方法，下面对象棋对弈的演化过程建立相应的数学模型[10]。图 1-1 给出了象棋博弈状态演化的过程图。

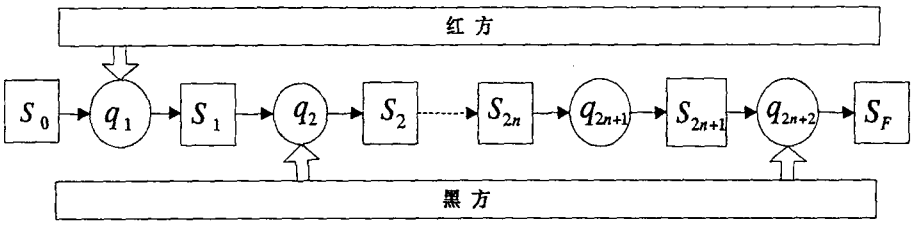


图 1-1 象棋博弈状态演化过程图

Figure 1-1 The evolutive process of chess game state

图中表明棋局状态是在着法算子作用下演化的，对应的状态转移方程为：

$$S_{n+1} = S_n \cdot q_{n+1}, \quad S_0 = S(0) \tag{1.1}$$

式中 S_0 为象棋的初始局面，q_{n+1} 为第 $n+1$ 步的着法算子，S_{n+1} 为走完第 $n+1$ 步后的棋局。将上式展开，

$$S_N = S_0 \cdot q_1 \cdot q_2 \cdot q_3 \cdot \cdots \cdot q_N = S_0 \cdot Q \tag{1.2}$$

式中，S_N 为终局，或红胜，或黑胜，或和棋。显然，着法序列 $Q = \{q_1 q_2 q_3 \ldots q_N\}$ 便是记录博弈过程的棋谱，单数项序列 $Q_{odd} = \{q_1 q_3 \ldots q_{2n+1} \ldots\}$ 为红方的着法序列，偶数项序列 $Q_{evn} = \{q_2 q_4 \ldots q_{2n+2} \ldots\}$ 为黑方的着法序列。

着法便是弈棋者的决策。尽管每次只能走一步棋，但弈棋者可能考虑过全部有价值的走法，并且通过前瞻若干步，形成弈棋者的当前决策。图 1-2 展示了这个“思维”过程的计算机实现。

中国知网 https://www.cnki.net

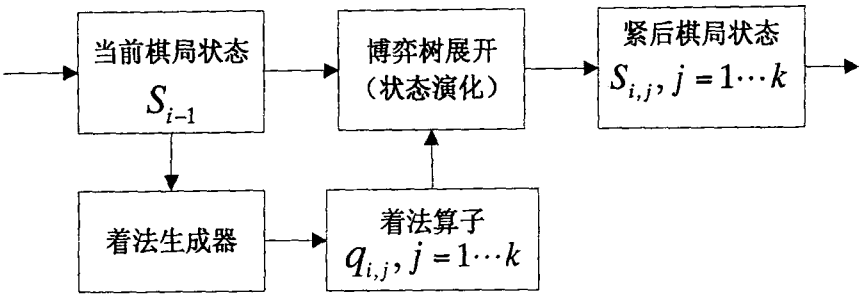


图 1-2 博弈者思维过程的机器实现流程

Figure 1-2 The flow chart of player's thinking process realized by computer

博弈过程中全部可能的走法是通过着法生成器给出的，由它生成了第i步的k种可行着法，根据状态演化方程(1.1)式，在算子$q_{i,j}, j=1..k$的作用下所形成的第i步后的棋局便不是一个，而同样是k个紧后棋局状态，即$S_{i,j}, j=1..k$，这样便展开了一棵博弈树，它与一般决策树的不同点则在于它的决策主体不是一方，而是相互对立的双方，即红方和黑方。

博弈树的每一个节点都代表一个棋局局面，棋手要在众多的叶子节点上挑选一个最佳的局面作为自己的选择，从而反推到当前的着法。中国象棋计算机博弈问题也就是寻求最佳路径的问题，它是通过博弈树搜索和评估函数来求解的。

1.2 数据结构和评估函数在中国象棋博弈中的作用

数据结构是计算机进行计算和存储棋盘信息的最基本条件。想让计算机学会下棋，首先必须解决的问题就是把棋盘信息编码到计算机中，转化为计算机可以处理的数据。良好的数据结构不仅能最大程度地方便着法生成，更是保证搜索和评估计算效率的基础。在计算机计算能力相同的情况下，相同的搜索引擎和评估函数，若使用好的数据结构，则能更快地产生着法，更快地对棋子关系进行评价，这样就更节约时间。没有良好的数据结构设计，庞大的博弈树的展开速度就会成为博弈系统性能的瓶颈，所以有效地构架一个适合中国象棋的数据结构是推动中国象棋计算机博弈发展的基础。

在计算机象棋博弈研究领域中，搜索算法和评估函数是一个博弈系统必须的，也是研究的核心部分。在实际的对弈中，由于博弈树极其庞大，无法穷尽博弈的一切可能性达到最终局面，搜索的深度是有限的，评估函数的工作就是对一定深度处的节点进行评分，用估计值代替实际的搜索，再根据该分值来反推当前各种着法的优劣，从而确定当前的博弈路径。评估函数的好坏直接影响到着法的选择，因而它在计算机博弈中有着非常重要的作用，主要有以下几点[11]：

(1) 评估函数是启发式搜索的重要组成部分。在中国象棋中，如果按照每一个棋局平均有 45 种可行着法，每局棋平均走 90 步计算，那从开始到分出胜负，则要考虑$45^{90} \approx 10^{150}$种可能性，据说这个天文数字比地球上原子数目还要多，即使用世界上最快的计算机进行计算，直到地球毁灭也无法算出第一步的着法[15]。可见，对完全博弈树的搜索是不可行的，必须在一定深度处停止搜索，对这个深度处的节点进行评分，用以作出判断，评估函

中国知网 https://www.cnki.net

数是决定整体搜索方向的最重要的判断手段。

(2) 评估函数是对局面评价的客观依据。所谓局面，就是对弈双方在棋盘上各方棋子的种类、数目、位置、相互关系以及对胜负的影响所构成的总和。只有使用了评估函数，计算机才能对局面进行精确客观的评估。

(3) 评估函数是对弈中计算机确定博弈走法的量度。在中国象棋博弈中，我们只有对一定搜索深度下的局面情况作出正确的判断，才能选择路径，找出合理的走法，否则计算机将无从选择。

(4) 评估函数的优劣，直接影响着博弈的结果。博弈双方根据自身的特点，确定自己的博弈策略和走法，在条件允许的情况下，谁对局面和整个局势的评估越准确，谁的搜索深度越深，那么谁就将在博弈中占有优势。

1.3 本文的主要工作

本文在阅读了国内外大量的国际象棋相关技术和资料的基础上，结合近年来中国象棋计算机博弈的发展实践，研究总结了中国象棋计算机博弈系统的关键技术，主要有下面四个部分：

(1) 数据结构：包括局面表示和着法生成；

(2) 博弈树搜索技术：主要是搜索算法，包括完全搜索、静态搜索、启发式搜索、剪枝搜索等；

(3) 评估函数：包括知识和优化，用以配合搜索引擎使计算机作出智能的选择；

(4) 开局库、残局库、后台思考等其他技术。

由于本课题的研究刚刚开始，之前没有任何研究基础，故本文主要针对如何从数据结构和评估函数的方面提高中国象棋博弈水平，设计了中国象棋计算机博弈的数据结构，并对评估函数的构建及优化进行了深入研究与实现，在此基础上开发了一个完整的博弈系统，该系统已应用于实战，具有业余棋手水平。本文的主要工作如下：

第一章介绍中国象棋计算机博弈技术的发展历程和发展现状，总结了数据结构和评估函数在计算机博弈中的重要作用，并给出了本文研究工作的目的和内容。

第二章主要研究使用什么样的数据结构来表示棋盘上的信息和加速着法生成，设计了适合中国象棋计算机博弈的编码方式，解决了如何把棋盘分布信息和棋盘状态信息编码进计算机的问题，并使用模板匹配和预置表法加快着法生成，为整个博弈系统的实现打下了良好的基础。

第三章介绍了博弈树搜索技术。简要介绍了博弈树求解的基本搜索算法——极大极小值算法和 a-ß 剪枝算法。

第四章主要探讨评估函数的构成和优化。首先对评估函数建立数学模型，在评估函数参数优化的研究中，针对目前中国象棋博弈引擎评估函数使用和优化的现状，分析介绍了手工调整估值参数和机器自学习优化参数的方法，以及提高评估函数准确性的一些有效的评估策略；提出了把棋子价值看作是全动态的，对其进行全动态评估的思想。在系统实现

中国知网 https://www.cnki.net

中主要采用手工方法优化估值参数，并在评估函数中加入了藐视因子和局势因子，有效地提高了对弈的获胜率。

第五章简略介绍了本文博弈系统的开发和实现。

第六章总结全文工作，讨论了中国象棋计算机博弈面临的问题和今后研究的重点。

中国知网 https://www.cnki.net

2 数据结构

中国象棋计算机博弈的数据结构即使用什么样的编码方式来表示棋盘上的信息以及如何方便快速地生成着法。要想让计算机下棋，首先需要解决的就是棋盘和棋子的编码表示，设计一种数据结构来表示棋局的状态，要考虑三个方面：占用空间的数量、操作速度、使用方便与否[12]。

数据结构是计算机博弈的基础，它包括局面表示、着法移动、着法生成等，对博弈树的展开与恢复、评估函数有着决定性的作用。

2.1 局面表示

中国象棋的棋盘是由10条横线、9条竖线构成的90个交叉点组成，红黑各7种共32个棋子，棋子位于交叉点上。根据走法规则红黑交替走棋对战，被吃掉的棋子要从棋盘上拿掉，以吃掉对方的将（帅）为游戏终止的判断。图2-1表示了中国象棋棋盘棋子的初始状态。

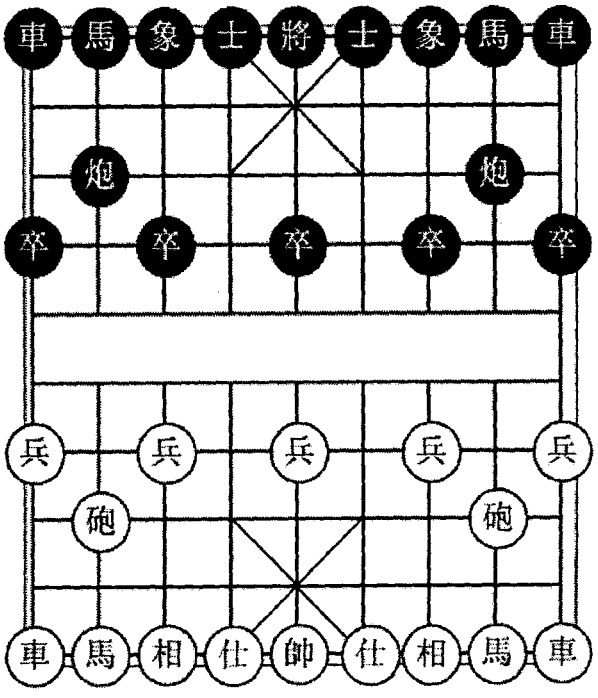


图2-1 中国象棋初始化棋盘

Figure 2-1 Initial chessboard of Chinese Chess

2.1.1 棋盘表示

对中国象棋棋盘的表示，最早是用10×9的二维数组，即10行9路，数组中每个字节代表棋盘上的一个交点，其值表示这个交点上放置的是什么棋子，或者没有棋子。表2-1给出了对棋盘坐标进行编码的二维数组，棋盘左上角的点就是 Board[0][0]，整个棋盘用Board[10][9]来定义。

中国知网 https://www.cnki.net

表 2-1 二维数组的棋盘坐标编码

Figure 2-1 Chessboard encoded by two-dimensional arrays

0,0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8
1,0	1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8
2,0	2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8
3,0	3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8
4,0	4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8
5,0	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8
6,0	6,1	6,2	6,3	6,4	6,5	6,6	6,7	6,8
7,0	7,1	7,2	7,3	7,4	7,5	7,6	7,7	7,8
8,0	8,1	8,2	8,3	8,4	8,5	8,6	8,7	8,8
9,0	9,1	9,2	9,3	9,4	9,5	9,6	9,7	9,8

二维数组的表示方法在查找时要做一次地址偏移操作，比较费时[13]，现在已经很少使用。还有一种棋盘表示方法是参照国际象棋棋盘8×8的表示方法，把棋盘做成16×16的大小，用 Board[256]数组对棋盘坐标编码，行号和列号可以通过 $N/16$ 和 $N\%16$ 计算得到，速度较快。编码表示如表 2-2 所示。

表 2-2 16×16的棋盘表示

Table 2-2 Chessboard denoted by 16×16 codes

00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e	1f
20	21	22	23	24	25	26	27	28	29	2a	2b	2c	2d	2e	2f
30	31	32	33	34	35	36	37	38	39	3a	3b	3c	3d	3e	3f
40	41	42	43	44	45	46	47	48	49	4a	4b	4c	4d	4e	4f
50	51	52	53	54	55	56	57	58	59	5a	5b	5c	5d	5e	5f
60	61	62	63	64	65	66	67	68	69	6a	6b	6c	6d	6e	6f
70	71	72	73	74	75	76	77	78	79	7a	7b	7c	7d	7e	7f
80	81	82	83	84	85	86	87	88	89	8a	8b	8c	8d	8e	8f
90	91	92	93	94	95	96	97	98	99	9a	9b	9c	9d	9e	9f
a0	a1	a2	a3	a4	a5	a6	a7	a8	a9	aa	ab	ac	ad	ae	af
b0	b1	b2	b3	b4	b5	6b	b7	b8	b9	ba	bb	bc	bd	be	bf
c0	c1	c2	c3	c4	c5	c6	c7	c8	c9	ca	cb	cc	cd	ce	cd
d0	d1	d2	d3	d4	d5	d6	d7	d8	d9	da	db	dc	dd	de	df
e0	e1	e2	e3	e4	e5	e6	e7	e8	e9	ea	eb	ec	ed	ee	ef
f0	f1	f2	f3	f4	f5	f6	f7	f8	f9	fa	fb	fc	fd	fe	ff

中国知网 https://www.cnki.net

这种方法对除车和炮以外的短程棋子的着法生成比较方便，因为它们的着法都有固定的增量(行的增量，列的增量)，处理起来比较简单。例如马有 8 个着法，增量分别是 ±0x0e、±0x12、±0x1f 和 ±0x21，红方的过河兵有 3 个着法，增量分别是 -0x10 和 ±0x01。判断棋子是否过河：红方是(SqFrom/SqTo & 0x80) != 0，黑方是(SqFrom / SqTo & 0x80) == 0，其中 SqFrom 和 SqTo 分别是棋子的起始位置和落址位置。另外，16×16的扩展棋盘加入了“出界”的标记，如表中外围深底色的表格所示，目标格在这些位置上就说明着法无效，这样就简化了边界检验的条件。

经过研究比较，考虑到存储空间的节省，并兼顾较快的操作速度，本文在设计博弈系统时采用 0~89 的一维数组对棋盘坐标进行编码，表示中国棋盘 90 个交叉点，如表 2-3 所示。

表 2-3 一维数组的棋盘坐标编码

Table 2-3 Chessboard encoded by one-dimensional arrays

00	01	02	03	04	05	06	07	08
09	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44
45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62
63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89

这种编码方法的数据操作是一维寻址，有速度上的优势。而且因为本文在着法生成上主要采用模版匹配、预置表等方法，只需要查询就可以生成着法，无需进行行号列号计算，这样就比16×16棋盘的速度还要快。并且采用 0~89 的棋盘比16×16的棋盘节省存储空间，所以选择 0~89 的一维数组是适合本文的实际的，也是有利于中国象棋对弈系统的实现的。下面对着法生成等技术的介绍和实现都是基于这种棋盘表示方式的。

2.1.2 棋子表示

中国象棋的棋子共有 7 种类型，区分红黑两种棋子，共计 14 种棋子，加上某个位置没有棋子的情况，总共是 15 种状态。本文采用-7~+7 对棋子种类进行编码，这种编码对于红黑对称的数据可以压缩处理，节省空间，表 2-4 给出了棋子种类编码。

中国知网 https://www.cnki.net

表 2-4 棋子种类编码

Table 2-4 Chessman-type encoding

黑将	黑车	黑马	黑炮	黑象	黑士	黑卒	无棋子
-1	-2	-3	-4	-5	-6	-7	0
红帅	红车	红马	红炮	红相	红仕	红兵	
1	2	3	4	5	6	7	

另外，中国象棋一共有 32 个棋子，棋子所处的位置是很重要的信息，如果只有棋子种类编码，要确定某个棋子的确切位置还需要进行大量的计算，为了解决这个问题，本文使用 0~32 对棋子个体进行编码，如表 2-5 所示。

表 2-5 棋子个体编码

Table 2-5 Chessman encoding

无棋子	黑将	黑车		黑马		黑炮		黑士		黑象		黑卒				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	红帅	红车		红马		红炮		红仕		红相		红兵				
	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32

2.1.3 棋盘分布信息表示

除了棋盘棋子的表示，还需要知道棋子在棋盘上的分布信息，而棋盘的分布信息是着法生成、评估函数以及搜索等必须的信息，本文采用棋盘索引数组、棋子索引数组和映射数组三者配合的方法，可以方便地表示棋盘分布信息。

a. 棋盘索引数组

棋盘索引数组是棋盘坐标编码（表 2-3）和棋子种类编码（表 2-4）的结合，是以数组的形式把实际的棋盘状态编码进电脑。它的数组坐标表示棋盘坐标编码的编号，即实际棋盘的某一位置，数组存储的内容是棋子种类编码的编号（表 2-4 所示），表示在实际棋盘上位于此位置的棋子的种类。

在中国象棋初始局面（图 2-1 所示）下，本文博弈系统使用的棋盘索引数组 ChessBoardIndex[90]初始化如下图所示：

中国知网 https://www.cnki.net

```
ChessBoardIndex[90] ={
    -2,  -3,  -5,  -6,  -1,  -6,  -5,  -3,  -2
     0,   0,   0,   0,   0,   0,   0,   0,   0,
     0,  -4,   0,   0,   0,   0,   0,  -4,   0,
    -7,   0,  -7,   0,  -7,   0,  -7,   0,  -7,
     0,   0,   0,   0,   0,   0,   0,   0,   0,
     0,   0,   0,   0,   0,   0,   0,   0,   0,
     7,   0,   7,   0,   7,   0,   7,   0,   7,
     0,   4,   0,   0,   0,   0,   0,   4,   0,
     0,   0,   0,   0,   0,   0,   0,   0,   0,
     2,   3,   5,   6,   1,   6,   5,   3,   2}
```

图 2-2 初始局面的棋盘索引数组

Figure 2-2 Chessboard index array of initial chessboard

b. 棋子索引数组

仅仅依靠上面的棋盘索引数组，相同种类的棋子用相同的编码，每次都必须遍历整个数组，进行棋子种类匹配，才能够确定某个棋子的确切位置，这给着法生成、棋局评估等带来不便。为了避免大量的费时操作，还需要设计棋子索引数组。

棋子索引数组是棋盘坐标编码与棋子个体编码的结合体，它记录了每个棋子在棋盘上的位置信息。与棋盘索引数组相反，棋子索引数组的坐标为棋子个体编码的编号（表 2-5 所示），存储的信息为棋子在棋盘上的位置，如果棋子被吃掉，则存储的是棋子不存在的标志 INVALID。

在中国象棋初始局面（图 2-1 所示）下，本文博弈系统使用的棋子索引数组 ChessmanIndex[33]初始化如下图所示：

```
ChessmanIndex[33]={
        INVALID,4,0,8,1,7,19,25,3,5,2,6,27,29,31,33,35,
        85,81,89,82,88,64,70,84,86,83,87,54,56,58,60,62};
```

图2-3 初始局面的棋子索引数组

Figure 2-3 Chessman index array of initial chessboard

c. 映射数组

棋盘索引数组和棋子索引数组在搜索过程中是同时存在的，这给棋盘的维护和更新带来了很大的困难。例如，某个棋子从 A 位置移动到 B 位置，利用棋盘索引数组可以方便的由位置信息完成更新，而棋子索引数组必须要遍历整个数组才能确定移动的是 0~32 编码中的哪个棋子，这很费时，为了达到快速高效的目的，设计一个映射数组[14]。

映射数组定义为棋盘索引数组和棋子索引数组的结合，也可以看成是棋盘坐标编码和棋子个体编码（表 2-5 所示）的另一种结合方式。图 2-4 是中国象棋初始局面下，本文博

中国知网 https://www.cnki.net

弈系统使用的映射数组 MappingChess[90]的初始化表示。

```
MappingChess[90]={
        2,  4,  10,  8,  1,  9,  11,  5,  3,
        0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  6,  0,  0,  0,  0,  0,  7,  0,
        12,  0,  13,  0,  14,  0,  15,  0,  16,
        0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,
        28,  0,  29,  0,  30,  0,  31,  0,  32,
        0,  22,  0,  0,  0,  0,  0,  23,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,
        18,  20,  26,  24,  17,  25,  27,  21,  19}
```

图 2-4 初始局面的映射数组

Figure 2-4 Mapping array of initial chessboard

当棋子移动时，根据映射数组，只需对棋盘索引数组和棋子索引数组进行很少的调整，就可以快速更新数据信息，也就是说，当局面发生变化时，棋盘索引数组和映射数组跟着更新，再把棋子和位置信息传递给棋子索引数组进行更新。显然，映射数组是棋盘索引数组和棋子索引数组之间的桥梁，图 2-5 是棋盘索引数组，映射数组和棋子索引数组三者之间的关系图。

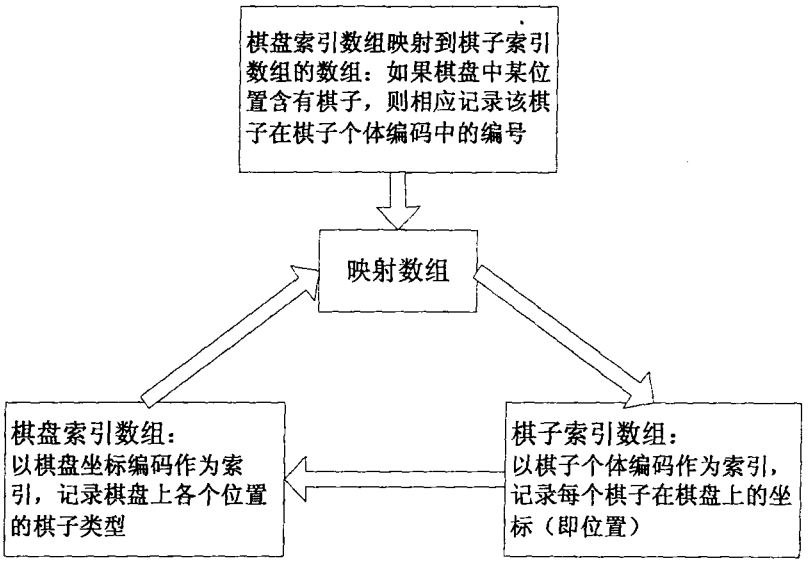


图 2-5 棋盘索引数组、棋子索引数组和映射数组的关系

Figure2-5 The relationship of chessboard index array, mapping array and chessman index array

在设计和实现博弈系统的数据结构时，很关键的一点是在搜索过程和棋局发展过程中，必须保证棋盘索引数组，映射数组和棋子索引数组三者同时更新。图 2-6 是本文设计的博弈系统中的一段三者同时更新的 C++程序片段，并定义如下结构：

中国知网 https://www.cnki.net

```
struct _chessmove{
    short    ChessID;    //表明是什么棋子
    char     From;       //起始位置
    char     To;         //目标位置
    int      Score;      //棋子的评估值
}ChessMove;
struct _chessUnmakemove
{
    char nchessID;    //棋子坐标
    char mappingID;   //映射数组坐标
}ChessUnMakemove;
ChessMove   * move;
ChessUnMakemove   chessUN;
```

```
ChessUnMakemove chessUN;
chessUN.nchessID = ChessBoardIndex[move->To]; //取目标位置
//把棋子移动到目标位置
//更新棋盘索引数组
ChessBoardIndex[move->To] = ChessBoardIndex[move->From];
ChessBoardIndex[move->From] = NOCHESS;
//更新棋子索引数组
ChessmanIndex[MappingChess[move->To]] = INVALID;
ChessmanIndex[MappingChess[move->From]] = move->To;
//更新映射数组
chessUN.mappingID = MappingChess[move - > To];
MappingChess[move->To] = MappingChess[move->From];
MappingChess[move->From] = NOCHESS;
```

图 2-6 棋盘索引数组，映射数组和棋子索引数组同时更新的程序片段

Figure2-6 The program of updating chessboard index array, mapping array and chessman index array synchronously

2.1.4 棋盘状态信息表示

对于车、炮这些棋子的着法生成或者将军判断、对脸判断等一些情况，需要大量的某行某列信息，只知道棋盘的空间分布信息还不够，还需要知道棋盘的空间状态信息，在本文的博弈系统设计中引入路向行向比特向量和比特棋盘这两种数据结构，能方便有效地得到这些信息，为快速生成着法提供了良好的辅助方法。

a. 路向行向比特向量（半位棋盘）

比特向量是使用二进制编码0或1表示的向量。中国象棋的车、炮、兵等棋子都是沿直

中国知网 https://www.cnki.net

线行走，需要大量的行列信息，将比特向量引入到中国象棋计算机博弈中有其先天的优势。路向行向比特向量也称为位行位列(BitRank, BitFile)，它是对棋盘的每一行，每一列用二进制编码0或1来记录该点上是否有棋子，这样每一行每一列都有一个编码值，其中行是$2^9=512$的编码空间，列是$2^{10}=1024$的编码空间。式(2.1)是用路向行向比特向量表示棋盘行和列的状态信息的表达式。

$$B=\begin{bmatrix} B_1^H \\ \vdots \\ \vdots \\ B_{10}^H \end{bmatrix}=\left[B_1^V \ldots\ldots B_9^V\right] \tag{2.1}$$

式中，B为当前局面的空间状态，B_i^H为行向比特向量，B_i^V为路向比特向量。定义两个整数数组$HORIZON[10]$和$VERTICAL[9]$分别表示棋盘上的所有行向和路向的棋子状态信息。对于初始局面（图2-1），初始化行向信息$HORIZON[10]$如下所示：

$$\begin{aligned}
&HORIZON[0]=\{1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\}=511\\
&HORIZON[1]=\{0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\}=0\\
&HORIZON[2]=\{0\ \ 1\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1\ \ 0\}=130\\
&HORIZON[3]=\{1\ \ 0\ \ 1\ \ 0\ \ 1\ \ 0\ \ 1\ \ 0\ \ 1\}=341\\
&HORIZON[4]=\{0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\}=0\\
&HORIZON[5]=\{0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\}=0\\
&HORIZON[6]=\{1\ \ 0\ \ 1\ \ 0\ \ 1\ \ 0\ \ 1\ \ 0\ \ 1\}=341\\
&HORIZON[7]=\{0\ \ 1\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1\ \ 0\}=130\\
&HORIZON[8]=\{0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\}=0\\
&HORIZON[9]=\{1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\ \ 1\}=511
\end{aligned} \tag{2.2}$$

对于初始局面（图2-1），初始化路向信息$VERTICAL[9]$如下所示：

$$\begin{aligned}
&VERTICAL[0]=\{1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\}=585\\
&VERTICAL[1]=\{1\ \ 0\ \ 1\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1\ \ 0\ \ 1\}=645\\
&VERTICAL[2]=\{1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\}=585\\
&VERTICAL[3]=\{1\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1\}=513\\
&VERTICAL[4]=\{1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\}=585\\
&VERTICAL[5]=\{1\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1\}=513\\
&VERTICAL[6]=\{1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\}=585\\
&VERTICAL[7]=\{1\ \ 0\ \ 1\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1\ \ 0\ \ 1\}=645\\
&VERTICAL[8]=\{1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\ \ 0\ \ 0\ \ 1\}=585
\end{aligned} \tag{2.3}$$

路向行向比特向量是同棋盘索引数组、棋子索引数组、映射数组同时存在的数据格式，随着搜索的进行，在树的展开和恢复时进行更新，不需要遍历整个棋盘。使用路向行向比

中国知网 https://www.cnki.net

特向量可以快速得到行列信息，利用微小的代价达到快速高效的目的。

b. 比特棋盘（位棋盘）

比特棋盘即位棋盘(BitBoard)，它使用二进制编码 0 或 1 来表示整个棋盘的状态，是由前苏联 KASSIA 小组于 60 年代提出的一种数据结构，最初应用于国际象棋。比特棋盘的作用就是记录象棋棋盘上的某些布尔条件，即如果比特棋盘中对应某一格的位是“1”，那么这一格上的条件就是“真”；如果是“0”则对应的条件就是“假”。布尔条件为自己所定义的条件，比如“棋盘哪些位置有棋子？”，“棋盘哪些位置有红棋棋子？”，“棋盘那些位置有黑车？”等等。

用数学表达式定义比特棋盘为：

$$B=\left[b_{i,j}\right]_{9\times10},\quad b_{i,j}=\begin{cases}1 & s_{i,j}\neq 0\\ 0 & s_{i,j}=0\end{cases} \tag{2.4}$$

式中，B 为当前局面的空间状态，$b_{i,j}$ 为第 i 行第 j 列的位值，$s_{i,j}$ 表示定义的布尔条件。

国际象棋的比特棋盘是用一个 64 位长度的变量记录棋盘上的某些布尔值，因为国际象棋棋盘有 64 个格子，所以用 64 位二进制数正好与之对应[14]。而中国象棋棋盘有 90 个交叉点，将比特棋盘引入中国象棋要做一些必要的修改。本文使用 96 位长度的变量来表示中国象棋棋盘状态，即用 3 个 int 型的整数（每个 int 型 32 位）表示整个棋盘的状态，高 6 位不使用。表 2-6 是中国象棋初始棋盘上所有红子的比特棋盘表示。

表 2-6 红方棋子的比特棋盘表示

Table 2-6 The red chessman denoted by BitBoard

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
1	0	1	0	1	0	1	0	1
0	1	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1

根据棋盘位置编码表 2-3，最左上角位置为 0，设定对应于比特棋盘变量的最低位，按行优先编码形成 96 位的比特棋盘（高 6 位未使用），则红方棋子初始位置所对应的 96 位数是：

（高位）000000 111111111 000000000 010000010 101010101 000000000

000000000 000000000 000000000 000000000 000000000（低位）

中国知网 https://www.cnki.net

在中国象棋计算机博弈中使用比特棋盘，最大的好处是进行区域判断，快速且高效，例如在本文的博弈系统设计中，判断“九宫”区域数组定义为：

```
//九宫区域判断
const BOOL InPalace[90]={
                0,  0,  0,  1,  1,  1,  0,  0,  0,
                0,  0,  0,  1,  1,  1,  0,  0,  0,
                0,  0,  0,  1,  1,  1,  0,  0,  0,
                0,  0,  0,  0,  0,  0,  0,  0,  0,
                0,  0,  0,  0,  0,  0,  0,  0,  0,
                0,  0,  0,  0,  0,  0,  0,  0,  0,
                0,  0,  0,  0,  0,  0,  0,  0,  0,
                0,  0,  0,  1,  1,  1,  0,  0,  0,
                0,  0,  0,  1,  1,  1,  0,  0,  0,
                0,  0,  0,  1,  1,  1,  0,  0,  0 };
```

对应的比特棋盘为：

（高位）000000 000111000 000111000 000111000 000000000 000000000

000000000 000000000 000111000 000111000 000111000（低位）

在使用时，只需进行简单的按位“与”操作就可以判断棋子是否位于“九宫”区域。

比特棋盘只需进行少量的位操作就能很容易地实现数据更新，还可以根据需要设计在搜索过程中需要的比特棋盘的个数，在博弈树传递的全局结构中，除了 90 个元素的一维数组外，一般还需要维持 10 个左右的比特棋盘。比特棋盘最大的缺点是坐标的提取，从整数中提取最高有效位，通过 MSB（最高有效位）运算才能够得到，3 个 32 位 int 型 MSB 的提取要比国际象棋的 64 位慢好几倍，大量的 MSB 操作会令程序运行速度缓慢。

图 2-7 给出了各种数据结构的关联。

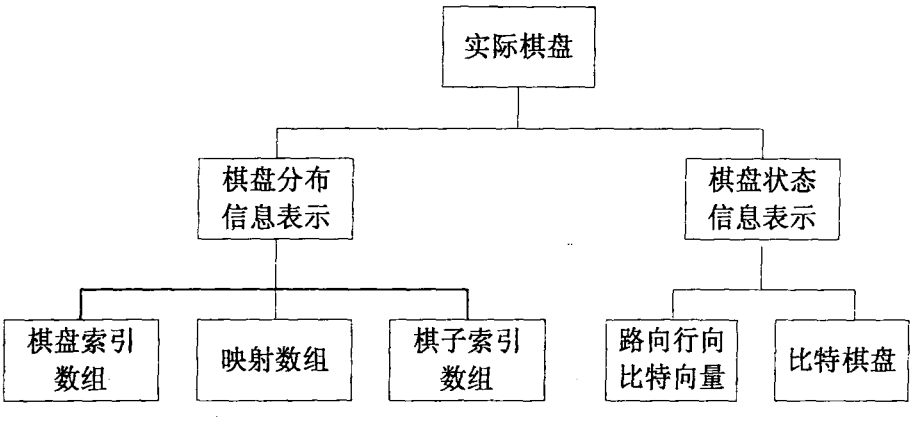


图 2-7 各种数据结构的关联

Figure 2-7 The relationship of kinds of Data Structure

2.2 着法生成

着法生成是指生成某一特定局面下走棋方所有可能的着法，也就是博弈树的展开。着法生成的快慢，即博弈树展开的快慢，对博弈程序的效率影响很大，着法生成和棋盘表示

中国知网 https://www.cnki.net

的数据结构是相辅相成的，着法生成的方法也决定了数据结构的形式。

除了表示局面信息的数据结构外，还需要设计一些可以加速着法生成的数据结构，本文使用的路向行向比特向量和比特棋盘就是很有效的辅助着法生成方法，另外模板匹配和预置着法表对着法生成也很有用。

2.2.1 模板匹配法

当动子确定以后，其“落址”和“提址”的相对关系便确定下来了，这样可以为某些动子设计其着法生成的“模板”，只要匹配到提址，便可以迅速找到落址。图 2-8 给出了马的匹配模板。

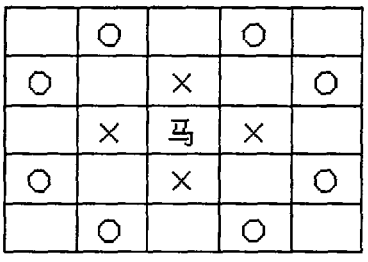


图 2-8 马的匹配模板

Figure 2-8 Template matching of Horse moving

将马对准提址，“○”表示符合“马走日”规则的落址，“×”表示“蹩马腿”的制约条件，通过查找模板匹配数组，实现“本方子则止，对方子则吃”，完成“提-动-落-吃”内容的确定。

下面是使用模板匹配法生成马的合法着法的 C++伪代码。

```
//产生马的合法走步
// SqFrom表示起始地址;
// j表示模板数组的内部循环变量;
// SqTo= HorseMove[SqFrom][j]表示符合马走日规则的落址;
// Leg = HorseLegs[SqFrom ][j]表示蹩马腿位置;
if (( ! IsSameSide(ChessBoardIndex[SqFrom],ChessBoardIndex[SqTo] ))
    &&ChessBoardIndex[Leg] == 0 )
     if (ChessBoardIndex[SqFrom]&&ChessBoardIndex[SqTo])   //判断有吃子否
         AddCapMove (SqFrom, SqTo, nPly);     //吃子着法
     else
         AddNoCapMove (SqFrom, SqTo, nPly); //非吃子着法
```

图 2-9 模板匹配法生成马的着法的伪代码

Figure 2-9 The pseudocode of template matching for Horse moving

由此可以看出，对马的着法生成使用模板匹配法，只要写出符合马行棋规则的模板匹配数组 HorseMove[90][10]和马腿的模板匹配数组 HorseLegs[90][10]，在生成着法时通过

中国知网 https://www.cnki.net

查找这两个数组，就可以实现。显然，这比扫描整个棋盘，通过计算得到合法落址要快速得多。同样的，可以对相（象）、象眼、将（帅）、士（仕）、兵（卒）使用模板匹配法生成着法。

2.2.2 预置着法表

预置着法表简称预置表，它是把所有可能的着法预先存储起来，在生成着法时，用查表取代计算。中国象棋每种棋子的行棋规则不同，生成每种棋子的着法和整个棋盘棋子的分布密切相关，如果建立每种棋子最大可能着法的小型数据库，用查询取代复杂的判断，那么也可以在很大程度上加快着法生成的速度。预置表看起来似乎很大，但是只需在程序开始运行时初始化一次就可以了，这些耗费对整个程序的影响不大，但是对着法生成的作用却是非常明显的。

在本文的博弈系统设计中使用了基于路向行向比特向量的预置表，图 2-10 即是以路向行向比特向量为索引的一个车的预置着法表的生成范例。图中显示，该车位于某行第 4 列，棋子分布的布尔向量形式为[010100100]，可以看出，此时车可能的吃子着法落址为[010000100]，非吃子着法的落址为[001011000]。把车的列号和该行棋子的布尔分布作为输入，把吃子落址和非吃子落址作为输出，将输入和输出的关系写入一个预置表中，在使用时通过查表，就可以快速构成可行着法，而且可以区分开吃子着法和非吃子着法。

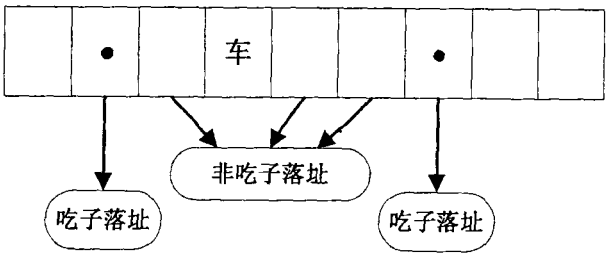


图 2-10 车的着法预置表范例

Figure 2-10 Prefabricated table for Rook

在系统实现时定义结构：

```
struct MoveStructTab {
    unsigned short NoCap, Rook_Attack, Cannon_Attack, SuperCannon;
    //车炮的位行位列：非吃子、车吃子、炮吃子、超级炮
    }MoveStructTab;
```

下面是使用路向行向比特向量预置表法生成车的着法的 C++伪代码。

中国知网 https://www.cnki.net

```
// 产生车的合法走步
// SqFrom表示起始位置
// nChessID = ChessBoardIndex[SqTo]; //棋子类型
// ChessX表示列号，ChessY表示行号
 MoveStructTab SqMv;
    //横向着法
    //j表示0～8内某列
    SqMv = RankMoveTab[ChessX][HORIZON[ChessY]];
        if(1<<j & SqMv.NoCap )
            AddNoCapMove ( SqFrom, SqTo, nPly ); //车的非吃子着法
        if(1<<j & SqMv.Rook_Attack )
            if(!IsSameSide(nChessID, ChessBoardIndex[SqTo]))
                AddCapMove(SqFrom, SqTo, nPly ); //车的吃子着法
    //纵向着法
    //j表示0～9内某行
    SqMv = FileMoveTab[ChessY][ VERTICAL[ChessX ]];
        if(1<<j & SqMv.NoCap )
            AddNoCapMove(SqFrom, SqTo , nPly); //车的非吃子着法
        if(1<<j & SqMv.Rook_Attack )
            if(!IsSameSide(nChessID, ChessBoardIndex[SqTo]))
                AddCapMove(SqFrom, SqTo, nPly ); //车的吃子着法
```

图 2-11 路向行向比特向量预置表生成车的着法的伪代码

Figure 2-11 The pseudocode of prefabricated table of bitrank and bitfile for Rook moving

2.2.3 测试实验研究

在模板匹配法和路向行向比特向量预置表实现时，本文对这两种方法进行了性能测试实验研究，实验的硬件条件是：Intel(R) Pentium(R) 4 CPU 2.60GHz , 2.59GHz 512MB 内存，WINDOWS XP 操作系统。

下表是使用模板匹配数据结构与不使用模板匹配的性能测试，表中数据是分别测试不同搜索深度的开中局 25 个回合总节点数和总耗时得到的。

表 2-7 模板匹配性能测试

Table 2-7 Capability testing of template matching

		不使用模板匹配	使用模板匹配	
搜索深度	节点数	耗时（ms）	耗时（ms）	节省时间(%)
1	10,187	852	634	25.59
2	37.026	1107	797	28.00
3	259.515	10356	6485	37.38
4	1,601,111	45988	20578	55.26

从上表可以看出，通过使用模板匹配数据结构可以快速生成着法，在搜索相同的节点

中国知网 https://www.cnki.net

个数的情况下，可以大幅度减少所需时间，并且随着搜索深度的增加，模板匹配的优势更加明显，可以节省用时50%以上。

表 2-8 是使用路向行向比特向量预置表数据结构与不使用路向行向比特向量预置表的性能测试。表中数据是分别测试不同搜索深度下，开中局25个回合的总节点数和总耗时得到的。

表 2-8 路向行向比特向量预置表性能测试

Table 2-8 Capability testing of prefabricated table of bitrank and bitfile

		不使用路向行向比特向量预置表	使用路向行向比特向量预置表	
搜索深度	节点数	耗时（ms）	耗时（ms）	节省时间(%)
1	10,187	634	612	3.47
2	37.026	797	743	6.78
3	259.515	6485	5336	17.72
4	1,601,111	20578	16121	21.66

从上表可以看出，通过使用路向行向比特向量的预置表数据结构，在搜索相同的节点个数的情况下，可以减少用时，并且随着搜索深度的增加，优势更加明显。

通过实验测试可以看出，在着法生成中使用模板匹配和预置表这两种数据结构可以有效的加快着法生成，提高系统性能。

2.3 小结

数据结构是计算机博弈的基础，通过实验可以看出，它对整个博弈引擎的性能（速度）影响巨大，对博弈树的展开与恢复、评估函数有着决定性的作用。数据结构使计算机可以进行下列操作：

(1) 执行一个着法(不仅是用户所选择的着法，而且包括在搜索过程中要用到的)；

(2) 撤消一个着法(作用同上)；

(3) 向用户显示棋盘；

(4) 产生所有可能的着法；

(5) 对棋盘的局面作出评估。

本章研究并设计了适合中国象棋计算机博弈的数据结构，包括局面信息的表示和加快着法生成的辅助数据结构。解决了如何把棋盘分布信息和棋盘状态信息编码进计算机的问题，模板匹配和路向行向比特向量预置表的数据结构的使用，有效的提高了博弈引擎的速度，为整个博弈系统的设计打下了良好的基础。

中国象棋计算机博弈的数据结构有多种选择，目前还没有形成统一或者公认为最佳的数据结构表示方式，究竟哪种方法更适合自己的引擎，决定于使用者的偏好取舍，考虑的角度不同选择也会不同，还需要进一步的实践和探索。但是否能准确、快速地统计棋盘棋子信息，是否能有效提高引擎速度是评价数据结构优劣的关键。

中国知网 https://www.cnki.net

3 博弈树展开与搜索

设想两人对弈的过程，我们将其中一位叫作甲，另一位叫作乙。假定现在该甲走棋，无论好坏甲可以有 40 种走法；而对甲的任意一种走法，乙也可以有与之对应的若干走法，乙走完棋后，又轮到甲走棋，对乙的走法甲又有若干种走法应对……，如此往复，直至分出胜负，对弈结束。

博弈树的定义 如果参加博弈的不是一个主体，而是对抗性的敌我双方，则一方在选择策略时不仅仅取决于自己的意愿，还要充分考虑对方应对的策略。由此产生的搜索树，通常称为博弈树，亦称决策树[4,8]。

显然，上面所述的两人对弈的情形就可以构建一棵博弈树。

3.1 博弈树的展开

几乎所有的棋类问题都可以用博弈树来描述[8]。博弈树的展开实际上就是把计算机和用户所有可能走法罗列出来，图 1-2 显示的博弈者思维过程机器实现流程中的博弈树展开就是这样一个过程。一个博弈树的根节点就是当前时刻的棋局，它的子节点是假设再走一步棋以后的各种棋局，孙子节点是从儿子节点的棋局再走棋一步后的各种棋局，依次类推，构造整棵博弈树直到可以分出胜负的棋局，弈棋者按照这样的思维方式，必然要展开一棵规模庞大的、根在上而叶在下的博弈树。图 3-1 给出了一个深度为 4 的博弈树的展开。

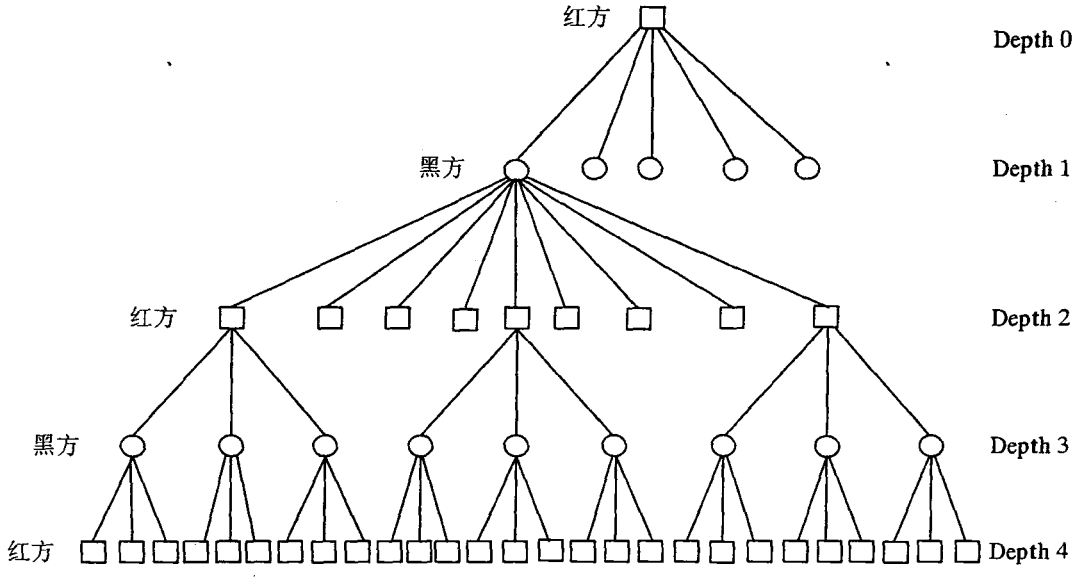


图 3-1 红方走棋时展开深度为 4 的博弈树

Figure 3-1 Four-depth game tree if the red one moves

图中方节点表示轮到红方走棋时的状态（棋局），圆节点表示轮到黑方走棋的状态。显然，方、圆节点间的连线就对应一个个着法。如果说一个优秀的棋手可以前瞻四步，则表示在他的脑海里将这棵博弈树展开四层(Depth)。博弈树上的每个节点都代表一个棋局，棋手要在众多的叶子节点上挑选一个最佳的局面作为自己的选择，再反推到当前的着法。

中国知网 https://www.cnki.net

3.2 博弈树搜索

从第一章的分析可以知道，中国象棋博弈树非常庞大，完全建立博弈树是不可能的，唯一的解决方法就是让博弈树扩展到计算机运算可以接受的深度，然后对没有分出胜负的叶子节点给出一个尽量准确的打分，表示此局面下取得胜利的可能性，这个打分是由评估函数计算给出的，将搜索树展开是着法生成的功能，而搜索引擎则是尽可能缩小树的规模，避免一切冗余的计算，这也是计算机博弈中搜索引擎研究的重点。

根据搜索策略，目前应用于计算机博弈的搜索算法[8,16]一般可分为三类：

(1) 穷尽搜索(Exhaustive Search)，这种搜索是没有抛弃任何可能成为最佳路径的搜索，不存在风险，得到的最佳走法肯定是在现有评估函数下应该得到的，例如极大极小值搜索[8,16-18]、a-ß 剪枝搜索[8,16,19]及其变种等。

(2) 选择性搜索(Selective Search)，即裁剪搜索，这种搜索略去对一些着法的搜索，冒着有可能漏掉最佳走法的风险，而换来局部更深的搜索深度。中国象棋中最常用的裁剪算法是空着裁剪(NullMove)等。

(3) 启发式搜索(Heuristic Search)[10,21-23]，利用象棋领域的知识（启发信息）设计搜索算法，着重对走法排序，以简化和加快搜索过程。中国象棋中的启发算法有历史启发、杀手启发、置换表等。

3.2.1 极大极小值搜索算法

极大极小值算法(Minimax Algorithm)是解决博弈树问题的基本方法。香农教授早在1950年首先提出了“极大极小算法”[3]，奠定了计算机博弈理论的基础。

极大极小值算法的原则是：博弈双方所要达到的目的相反，一方要寻找的利益是另一方失去的利益，博弈的一方总是希望下一步是子节点中取值最大者，而另一方则希望下一步是子节点中取值最小者。在象棋博弈中，极大极小值算法体现在始终站在博弈一方的立场上给棋局估值，有利于这一方的棋局给予一个较高的价值分数，不利于这一方（有利于另一方）的给予一个较低的价值分数，双方优劣不明显的局面给予一个中间值分数。在这一方走棋的时候，选择价值最大的子节点走法，即实行“Max 搜索”，另一方走棋则选择价值最小的子节点走法，即实行“Min 搜索”，这就是象棋博弈中的一个极大极小过程。

例如，如果红方为走棋方，它在偶数层的着法选择是要在其全部子节点中找到评估值最大的一个，实行“Max 搜索”，称为 MAX 方，而其敌对方即黑方在奇数层的着法选择则是在其全部子节点中要找到评估值最小的一个，实行“Min 搜索”，称为 MIN 方。图3-2表示了一个极大极小搜索过程，粗箭头为最佳路径片段。

中国知网 https://www.cnki.net

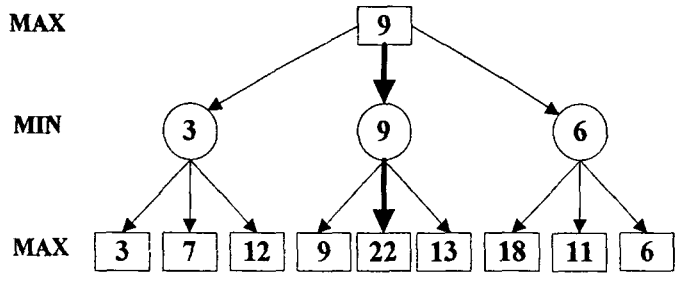


图 3-2 极大极小值算法示意图

Figure 3-2 The fig. of Minimax Algorithm

由于完整的博弈树过于庞大，程序不能也没有必要搜索整棵博弈树的所有节点，而需要像人类棋手一样有选择性地进行搜索，对于一些已经确定不佳的走步可以将以它为根节点的子树剪掉(cut-off / pruning)，以提高单位时间内搜索的节点数。

3.2.2 a-ß 剪枝搜索

上述极大极小值搜索过程中，遍历了整个的博弈树，这样的搜索算法效率低，搜索量非常大，如果要减少搜索量，又可能影响搜索效果。但假如将叶节点的评估计算与树的展开同时进行，然后对博弈树进行必要的裁剪，就可能大量减少所需搜索的节点数目，并且保持搜索效果不变。这个技术叫 a-ß 剪枝搜索(a-ß cut-off)。

a-ß 剪枝搜索是一种基于 a-ß 搜索的深度优先搜索(Depth-first search)，是所有剪枝算法的基础，它是根据极大极小搜索规则进行的。下面给出两个剪枝示意图：

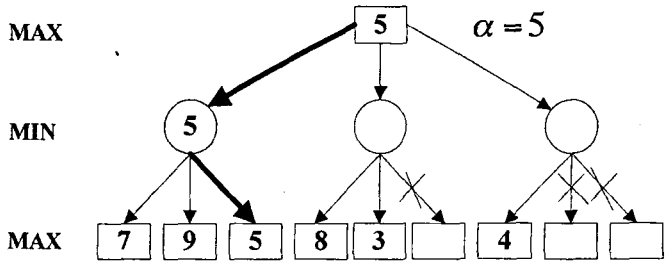


图 3-3 α 剪枝示意图

Figure 3-3 The fig. of a cut-off

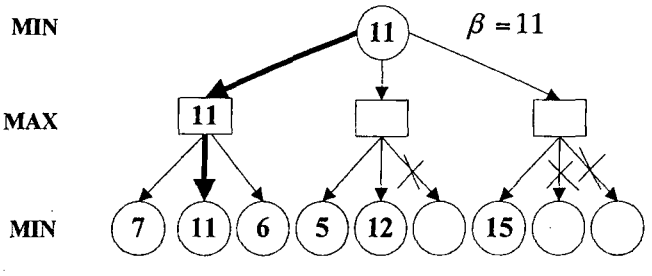


图 3-4 β 剪枝示意图

Figure 3-4 The fig. of ß cut-off

在图 3-3 所示的极大极小树片段中，按照极大极小值搜索规则，从左路分枝的叶节点倒推得到第一层 MAX 节点的值为 5，可表示此时的着法最佳值，记为 a，显然此 a 值可

中国知网 https://www.cnki.net

作为 MAX 方着法指标的下界。在搜索中路分枝时，因为第二层着法的选择是取第三层节点的最小值，即取 MIN(8, 3, "□")，而无论"□"中为何值，都不会比 5 大（最大为 3），故可以将"□"表示的节点及其后继节点剪掉，不再考虑此节点的延伸。同理搜索右路分枝，也可以进行剪枝操作。此类剪枝称为 a-剪枝。图 3-3 中通过剪枝，最后得到如粗箭头所示的最佳路径片断。

图 3-4 所示的极大极小树片段中，由左路分枝的叶节点倒推得到第一层 MIN 节点的值为 11，可表示此时对方着法的钳制值，记为 ß。显然此 ß 值可作为 MIN 方可能实现着法指标的上界。在搜索中路分枝时，因为第二层着法的选择是取第三层节点的最大值，即取 MAX(5, 12,"○")，而无论"○"中为何值，都不会比 11 小（至少为 12），故可以将"○"表示的节点及其后继节点剪掉，不再考虑此节点的延伸。同理搜索右路分枝，进行剪枝操作。此类剪枝称为 ß 剪枝。图 3-4 中通过剪枝，最后得到如粗箭头所示的最佳路径片断。

a-ß 剪枝搜索能够让我们省略许多节点的搜索，不只是节点本身，而且包括节点下面的子树，这样 a-ß 剪枝搜索需要遍历的节点数远少于极大极小算法所遍历的节点，也就节省了大量的时间开销，并且仍不失为穷尽搜索的本性。但 a-ß 剪枝的剪枝效果即搜索节点数与博弈树节点的搜索次序密切相关，从下面两个剪枝示例可以看出。

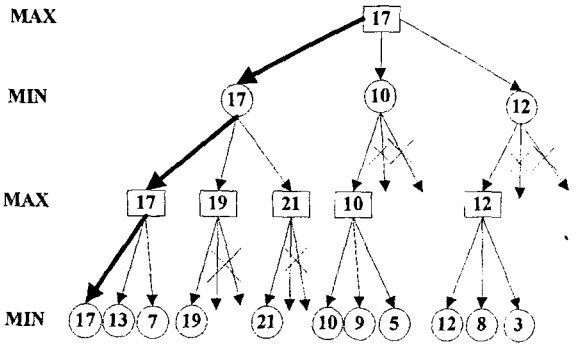


图 3-5 a-ß 剪枝的最佳情况

Figure 3-5 The best case of a-ß search

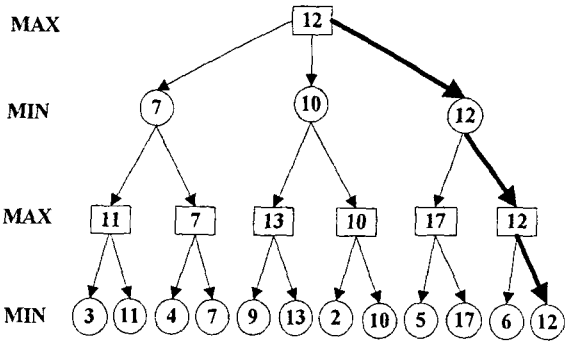


图 3-6 a-ß 剪枝的最差情况

Figure 3-6 The worst case of a-ß search

图 3-5 和图 3-6 分别显示了搜索深度为 3 时，a-ß 剪枝的最理想情况和最差情况。1975 年 Knuth 和 Moore 给出了在最理想的情况下（极小树），a-ß 剪枝算法的节点数目为：

$$\begin{aligned} N_D &= 2B^{D/2} - 1 \qquad (D\text{为偶数}) \\ N_D &= B^{(D+1)/2} + B^{(D-1)/2} - 1 \ (D\text{为奇数}) \end{aligned} \tag{3.1}$$

而在 a-ß 剪枝最差的情况下（极大树），需要搜索的节点数为：

$$N_D = B^D \tag{3.2}$$

其中 D 为搜索深度，B 为分枝因子。由式(3.1)和式(3.2)可以看出，最理想情况下 a-ß 剪枝搜索深度为 D 需遍历的节点数相当于最差情况下需遍历的节点数的平方根的两倍左右，也可以说，在相同的时间内，使用 a-ß 剪枝搜索的深度是不使用剪枝情况下的 2 倍。由式(3.1)计算图 3-5 中需要搜索的节点数为 $N_D = 3^{(3+1)/2} + 3^{(3-1)/2} - 1 = 11$，而在图 3-6 这种情况

中国知网 https://www.cnki.net

下，a-ß 剪枝搜索就相当于极大极小值搜索，显然 a-ß 剪枝就没有什么优势了。

3.2.3 其他搜索算法

极大极小值搜索是所有搜索算法的基础，而 a-ß 剪枝搜索是所有剪枝算法的基础。但在实际的博弈系统设计中，要提高引擎的搜索速度和搜索深度，还必须利用搜索中得到的一些启发式信息来加快搜索，目前已经有很多对基本搜索算法的增强算法。

在 a-ß 剪枝搜索中，剪枝效率几乎完全取决于节点的排列顺序，如何调整待展开的走法的顺序，是提高搜索效率的关键。在搜索的过程中，往往有一些启发性的信息可以利用，如果利用这些信息在相似的局面下对走法进行优化排序，或者对相同的局面不再搜索，而直接返回以前搜索的结果，都会对搜索产生明显的加速作用。围绕着法排序，已经出现了许多优秀的搜索算法与举措，例如历史启发，杀手启发，置换表等等。

历史启发(History Heuristic)实际上就是记录搜索过的好的走法，并在后续着法中优先搜索。根据经验，一些以前经过搜索认为最佳的走法，其后续走法仍然有很大可能成为最优的走法。在搜索过程中，每找到一个好的着法，就将与该走法相应的历史得分作一个增量，一个多次被搜索并确认为好的走法的历史记录分值会较高，当搜索中间节点时，将走法根据其历史得分排序，以获得较佳的排列顺序。

杀手启发(Killer Heuristic)是基于这样的思想：在搜索过程中，有许多走法一经搜索就引发了剪枝，且这些走法通常是同一个走法，这样，为每一层记录引发剪枝最多的走法即杀手走法，当下一次搜索到同样的深度时，如果杀手走法是该局的一个合法走法，就优先搜索杀手走法。杀手启发是一种特殊的历史启发，它不依赖于人类的知识，具有普遍性。

置换表(Transposition Table)是用一张表把搜索过的信息记录下来，在后续的搜索中，察看记录在表中的这些信息，如果将要搜索的某个节点已经有记录，就直接利用记录下来的结果引入到当前的搜索中，以减少搜索。置换表不同于历史启发表和杀手表，不必在每次搜索之前都清空，而是一直维持这些数据，作为以后搜索的信息。

另外，空着搜索(NullMove)[24-27]也是搜索算法中一种很有效的搜索策略，它的思想是放弃本方的走棋权利，让对方连续走棋，如果得到的分数还大于原来的 ß 值，就可以简单地返回 ß 值，在此分枝下的搜索意义不大，免于搜索。空着搜索明显降低了搜索的数量，导致搜索深度显著提高，并且危险性比较小，实现较为简单。

3.3 小结

搜索引擎是计算机博弈中涉及算法最多的模块之一，其搜索效率是众多搜索算法总的功能体现。搜索算法种类繁多，没有任何一种单一的算法能使搜索引擎达到很高的效率，往往是多种算法的结合，而各种增强手段之间既可能相互优化，又可能相互冲突、重复搜索浪费时间，所以搜索引擎必须要有一个良好的结构。本文的博弈系统在搜索引擎部分选择的组合方案是：以 a-ß 剪枝搜索为基础，用历史启发、置换表和静态启发算法进行优化，并使用空着裁剪作为增强手段，虽然这不一定是最快的方案，但为后续研究打下了良好的

中国知网 https://www.cnki.net

基础。

带有风险的裁剪也可以提高搜索效率，例如空着前向裁剪，可以很方便地利用很小的代价增加搜索深度 2~3 层，但是对于将军局面、无等着局面等特殊局面，若不做合理的判断往往会给搜索带来非常危险的判断，因而保证搜索的稳定性是搜索算法研究的重点之一。另外，在设计搜索引擎时不能只局限于搜索节点的数量上，因为搜索引擎只是整个程序效率的一个体现，搜索的节点少了，但漏掉了一些好的着法，导致棋力下降，是得不偿失的。如何更快更安全地搜索到最佳结果，是中国象棋计算机博弈领域需要更深入研究的课题。

中国知网 https://www.cnki.net

4 评估函数

对博弈树求解除了要有良好的搜索算法，另一个重要的方面就是评估函数。只有建立了良好的评估函数才能对局势作出客观的判断，保证较快的找到最佳走法。棋局评估函数就是对一定搜索深度处的棋盘局面评分，判断局面的优劣，使搜索引擎在搜索过程中依靠其值的大小而决定节点的取舍，帮助计算机作出正确的选择。同搜索算法一样，评估函数也是计算机博弈系统最核心的部分，它的好坏影响着今后局势的发展趋势。如果说搜索引擎是让程序看得更远，那么评估函数则是让程序看得更准。

棋局好坏的判断是一件非常复杂的事情，评估函数在很大程度上依赖于具体的游戏规则和我们对该游戏的经验知识，因此评估函数是与人类经验知识结合最密切的部分，而人类经验常以感性形式存在，在计算机的实现中，必须将其理性化，也就是建立评估函数的数学模型，并且对其参数进行优化，以便使计算机对局势进行合理、准确的判断。

4.1 评估函数与博弈性能

在博弈系统的几个主要部分里，评估函数是与具体的棋类知识紧密结合的一部分，可以说评估函数在很大程度上决定了博弈程序的棋力高低。

中国象棋博弈程序与国际象棋一样可以分为两类，即知识型程序和速度型程序。知识型是指赋予评估函数大量的象棋知识，用复杂的评估函数来弥补搜索算法的不足，在同等速度下，知识量越大性能越高；速度型是指采用简单的评估函数以获得更深的搜索深度，以速度取胜，在同等的知识含量下，速度越快搜索的深度越深，性能越高。图 4-1 表示了速度与知识的关系。

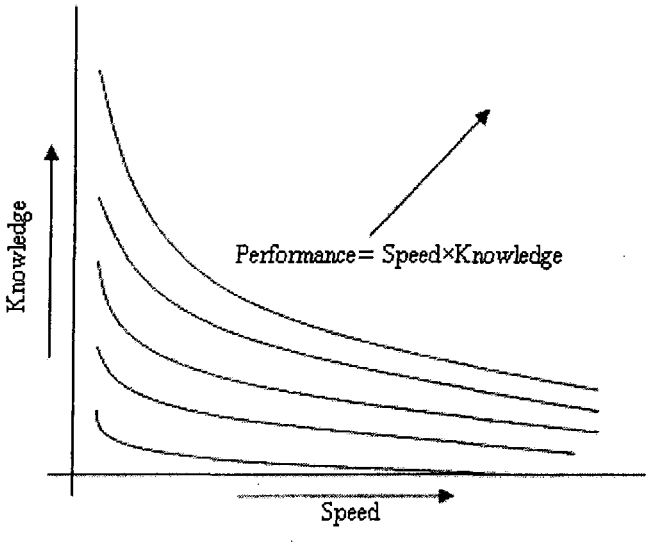


图 4-1 知识与速度的关系

Figure 4-1 The relationship of knowledge and speed

很显然，知识量和速度是一对成反比的量，而程序的棋力则是二者之积[28]，即：

Performance（性能）=Speed（速度）×Knowledge（知识）

中国知网 https://www.cnki.net

这一公式具有理论上的意义，它表明过于复杂的评估函数和过于简单的评估函数同样不佳。在速度和知识量的相互作用下，达到两者之间的平衡，是程序设计者追求的目标。但通常情况下很难确定评估函数应具备多大的知识量才合适，设计者需要通过大量的试验，来确定要将哪些知识放进评估函数。

4.2 评估函数建模

由于中国象棋的复杂性和灵活性，构建评估函数必须考虑多方面的因素以进行综合评判[16,29]，每个棋子的子力值、灵活度、处在不同位置对局势的影响，以及各棋子之间的相互关系等，都是要考虑的因素[12,32]。将这些感性知识理性化，就要建立评估函数的数学模型。

4.2.1 固定子力值的评价

子力是指棋盘中的每个棋子，根据它对局势的重要性和走法的丰富程度赋予其一个固定值。子力值是衡量局面优劣的首要因素，一般而言，子力多者占优，子力少者占劣。例如，根据我们的经验，车的战斗力最强，给定它的价值比马和炮的分值和略高；炮和马的价值大体相当，大约相当于士和相的和；士和相的价值相当，相当于双兵，等等。另外，在中国象棋博弈中，由于一般以将死对方的帅（将）作为对弈的结束，因此，将帅（将）的子力值赋为无穷大（通常用一个远大于其他棋子子力和的数值）。

由于棋手选择的策略和习惯不同，对每个棋子的认识也不同，在这些主观因素影响下，对子力的估值必然会有一定的差异，同样是一个卒，在高手眼里的作用不次于马、炮，因此对子力的评估要充分考虑算法和程序的特点。通过试验，本文设定棋子的固定子力值如表 4-1 所示。

表 4-1 各棋子的固定子力值

Table 4-1 The value of each chessman

棋子类型	车	马	炮	相（象）	仕（士）	兵（卒）	帅（将）
分值	950	450	450	200	200	130	65536

对单方棋子子力评估值的计算可以表示如下，

$$e_1(p) = \sum V_i \times n_i(p) \tag{4.1}$$

式中，V_i 表示类型为 i 的棋子固定子力值，如表 4-1 所示；$n_i(p)$ 表示该子在当前局面 p 上的数目；$e_1(p)$ 表示当前局面 p 上单方棋子子力之和。

4.2.2 棋子位置值的评价

棋子在棋盘中是否占据有利位置是评估函数中非常重要的信息，同样的棋子在不同位置上起的作用是不同的[16]，一般认为棋子越接近对方“九宫”对棋子的控制越强，对对方的威胁就越大，分值越高。例如，兵（卒）过河前后的攻击力以及对对方的威胁显然是

中国知网 https://www.cnki.net

不一样的，特别是进入对方“九宫”的兵的价值有时不一定比车少；车在不同的位置对对方棋子的威胁也不同，一个始终蹲在角落的车对局面控制起不到什么作用；马在己方可以说对对方没有什么威胁，但在对方“九宫”附近则会对对方帅（将）构成严重威胁；炮在中央位置或者和对方帅（将）在同一直线上时，会严重威胁对方。

另外，同一个棋子在开中局和残局时对局面的影响也是不同的，比如残局的兵如果运用得当，甚至可以将死对方，中局过河兵和残局兵的数量非常重要。

这些信息在评估函数中都应该有所体现，应该根据棋子所处的位置对局势的影响，以及对对方棋子的威胁程度赋予棋子一定的位置分值。图 4-2～图 4-7 分别是本文在设计评估函数时设定的红方的兵、车、马在开中局和残局设定的位置值，这些值是根据经验再通过对弈不断调整而确定的，能比较客观地反映子力在不同位置时对局面的影响。而要使用黑方棋子的位置值时，只需把对应的红方棋子的位置值数组以过河线为轴做镜像就可以得到。

```
R_PawnMidgame[90]={
    1,   3,   9,  10,  12,  10,   9,   3,   1,
   18,  36,  56,  95, 118,  95,  56,  36,  18,
   15,  28,  42,  73,  80,  73,  42,  28,  15,
   13,  22,  30,  42,  52,  42,  30,  22,  13,
    8,  17,  18,  21,  26,  21,  18,  17,   8,
    3,   0,   7,   0,   8,   0,   7,   0,   3,
   -1,   0,  -3,   0,   3,   0,  -3,   0,  -1,
    0,   0,   0,   0,   0,   0,   0,   0,   0,
    0,   0,   0,   0,   0,   0,   0,   0,   0,
    0,   0,   0,   0,   0,   0,   0,   0,   0}
```

图 4-2 红兵的开中局位置值

Figure 4-2 Position value of a red Pawn in open-midgame

```
R_PawnEndgame[90]={
    5,  10,  18,  25,  30,  25,  18,  10,   5,
   28,  55,  70, 115, 130, 115,  70,  55,  28,
   22,  50,  65,  85, 110,  85,  65,  50,  22,
   18,  25,  30,  56,  72,  56,  30,  25,  18,
    8,  12,  15,  20,  30,  20,  15,  12,   8,
    5,   0,   8,   0,  10,   0,   8,   0,   5,
   -5,   0,  -7,   0,  -1,   0,  -7,   0,  -5,
    0,   0,   0,   0,   0,   0,   0,   0,   0,
    0,   0,   0,   0,   0,   0,   0,   0,   0,
    0,   0,   0,   0,   0,   0,   0,   0,   0}
```

图 4-3 红兵的残局位置值

Figure 4-3 Position value of a red Pawn in endgame

中国知网 https://www.cnki.net

从图 4-2 和图 4-3 可以看出，兵过河后位置分值明显增大，进入对方“九宫”的兵价值最高，但当兵走到底线后，因为只能向左或向右走棋，位置值又会降低。而给定残局兵比开中局的兵高的位置值，是为了体现残局兵更具有攻击性。

```
R_RookMidgame[90]={
    185,  195,  190,  210,  220,  210,  190,  195,  185,
    185,  203,  198,  230,  245,  230  198,  203,  185,
    180,  198,  190,  215,  225,  215,  190,  198,  180,
    180,  200,  195,  220,  230,  220,  195,  200,  180,
    180,  190,  180,  205,  225,  205,  180,  190,  180,
    155,  185,  172,  215,  215,  215,  172,  185,  155,
    110,  148,  135,  185,  190,  185,  135,  148,  110,
    100,  115,  105,  140,  135,  140,  105,  115,  100,
    115,   95,  100,  155,  115,  155,  100,   95,  115,
     20,  120,  105,  140,  115,  140,  105,  120,   20}
```

图 4-4 红车的开中局位置值

Figure 4-4 Position value of a red Rook in open-midgame

```
R_RookEndgame[90]={
    160,  160,  160,  175,  195,  175,  160,  160,  160,
    175,  175,  175,  195,  205,  195,  175,  175,  175,
    160,  160,  160,  175,  195,  175,  160,  160,  160,
    145,  145,  145,  160,  175,  160,  145,  145,  145,
    145,  145,  145,  160,  175,  160,  145,  145,  145,
    145,  145,  145,  160,  175,  160,  145,  145,  145,
    120,  120,  120,  145,  165,  145,  120,  120,  120,
    115,  115,  115,  125,  165,  125,  115,  115,  115,
    115,  115,  115,  125,  170,  125,  115,  115,  115,
    100,  100,  100,  125,  165,  125,  100,  100,  100}
```

图 4-5 红车的残局位置值

Figure 4-5 Position value of a red Rook in endgame

从图 4-4 和图 4-5 可以看出，车位于对方棋盘内时位置值高些，而位于己方角落时位置值较低；开中局对弈双方展开激烈交战，车是主力，比残局时作用更大。

中国知网 https://www.cnki.net

```
R_HorseMidgame[90]={
    80,  105,  135,  120,   80,  120,  135,  105,   80,
    80,  115,  200,  135,  105,  135,  200,  115,   80,
   120,  125,  135,  150,  145,  150,  135,  125,  120,
   105,  175,  145,  175,  150,  175,  145,  175,  105,
    90,  135,  125,  145,  135,  145,  125,  135,   90,
    80,  120,  135,  125,  120,  125,  135,  120,   80,
    45,   90,  105,   90,  110,   90,  105,   90,   45,
    80,   45,  105,  105,   80,  105,  105,   45,   80,
    20,   45,   80,   80,  -10,   80,   80,   45,   20,
    20,  -20,   20,   20,   20,   20,   20,  -20,   20}
```

图 4-6 红马的开中局位置值

Figure 4-6 Position value of a red Horse in open-midgame

```
R_HorseEndgame[90]={
    90,  105,  135,  135,  135,  135,  135,  105,   90,
   105,  135,  165,  165,  165,  165,  165,  135,  105,
   135,  165,  200,  200,  200,  200,  200,  165,  135,
   135,  165,  200,  200,  200,  200,  200,  165,  135,
   135,  165,  200,  200,  200,  200,  200,  165,  135,
   105,  135,  165,  165,  165,  165,  165,  135,  105,
    85,  105,  125,  125,  125,  125,  125,  105,   85,
    70,   85,  105,  105,  105,  105,  105,   85,   70,
    30,   55,   85,   55,   55,   55,   85,   55,   30,
    25,   30,   70,   30,   30,   30,   70,   30,   25,
```

图 4-7 红马的残局位置值

Figure 4-7 Position value of a red Horse in endgame

图 4-6 和图 4-7 反映了马在开中局和残局的不同位置对局面的影响力。马在己方除了可以保护己方的棋子外起不到什么作用，但进入对方棋盘尤其是威胁到对方将时位置值明显增大。

由此，可得到计算单方位置评估值的数学表达式为，

$$e_2(p) = \sum W_{ij}(p) \times \lambda_i(p) \tag{4.2}$$

式中，$W_{ij}(p)$ 表示在局面 p 中一个棋子 i 处在 j 位置（j 表示棋盘0~89的某个位置）对应的位置分数值；$\lambda_i(p)$ 是棋子间位置值的权重；$e_2(p)$ 表示在 p 局面下单方总的位置评估值。

4.2.3 棋子灵活度的评价

棋子的灵活性是指棋子在该位置的活动范围，即向各处调动的可能性，棋子的威力能否充分发挥与它的灵活性直接相关，通常是越大越好。尤其是车、马、炮的灵活度对局面的影响很大，例如车在从直线上遇到障碍物，马被周围棋子绊马腿等，都降低了灵活度，

中国知网 https://www.cnki.net

也限制了棋子威力的发挥。

棋子灵活度的计算就是将一个棋子在棋盘上所有合法着法列出，乘上每一个可走步的价值[13]，那么局面 p 下单方灵活度评价表达式为，

$$e_3(p) = \sum H_i \times m_i(p) \tag{4.3}$$

式中，H_i 表示类型为 i 的棋子的灵活度系数，即棋子每多走一步加上的价值；$m_i(p)$ 表示当前局面 p 下该棋子可走的着法数；$e_3(p)$ 表示局面单方总的灵活度评价值。

对于灵活度系数的确定，考虑实战经验，兵（卒）的移动位置最少，但每移动一步价值增加明显，如图 4-2 和图 4-3 兵的位置得分所示，所以灵活度系数最高；马可移动的位置一般不多，但对对方的威胁往往较大，所以赋予马的灵活度系数次之；车炮的移动位置较多，但是这些位置往往对于棋局的意义相近，考虑棋子本身对对方的威胁，赋予车炮中间的灵活度系数；相（象）和仕（士）仅作为守子，对对方威胁不大，并且走法有很大的限制，赋予它们很小的灵活度系数；而帅（将）是棋局的灵魂，本身价值就无穷大，所以灵活度系数设为 0。考虑这些因素，在本文的评估函数中具体分配各棋子的灵活度系数如表 4-2 所示。

表 4-2 各棋子的灵活度系数

Table 4-2 Flexible value of each chessman

棋子类型	车	马	炮	相(象)	仕(士)	兵(卒)	帅(将)
灵活度系数	7	13	7	1	1	15	0

4.2.4 棋子的协调性、威胁与保护

在中国象棋博弈中，每个棋子都不是孤立存在的，它们之间构成了各种相互关系。棋子的协调性是棋子在该位置与己方其它棋子联系紧密的程度，如果棋子在该位置导致数个棋子（相同或不同类型）有目的的配合作战，服务于特定的作战目的，那么这样的棋子就有较好的协调性，应该给予一定的附加值。

棋子间的牵制是体现棋子协调性的一个重要方面。牵制往往被许多开发者所忽视，但是我们经过研究觉得牵制也是构建评估函数的一个研究重点。假如车马在未过河的时候就已经被对方的车在同一条线上牵制住，那么就会在局部上处于不利的形势。另外，“超级炮”，即炮隔两子吃子的方式，也是牵制中一个很重要的例子，如果程序可以认识到这种情况，对棋力的提高也是很有帮助的。下面以图 4-8 的局面为例，介绍牵制的作用。

中国知网 https://www.cnki.net

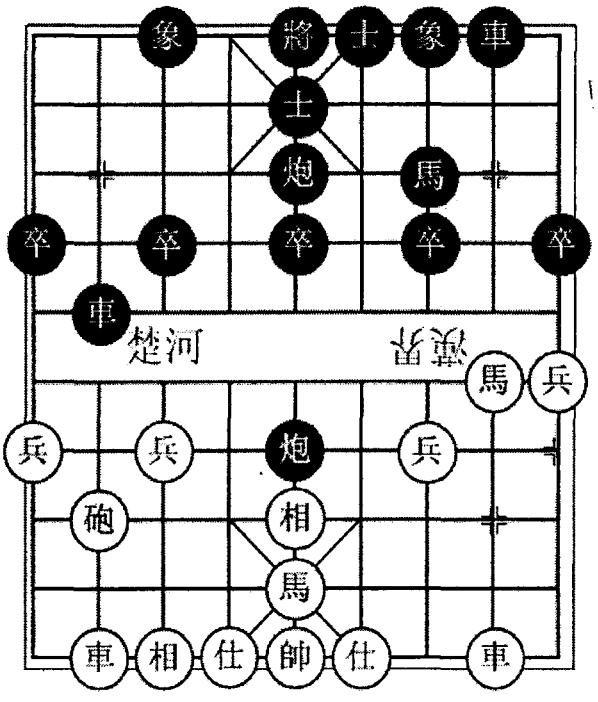


图 4-8 棋子间的牵制举例

Figure 4-8 Example of containing between chessman

在这个局面中，红方的“车”（第 10 行第 2 列）和“马”（第 6 行第 8 列）在未过河的时候就已经被黑“车”（第 5 行第 2 列和第 1 行第 8 列）牵制住，那么就会在局部上处于不利的形势；而黑“炮”（第 7 行第 5 列）形成“超级炮”棋型，如果不是被红“相“（第 8 行第 5 列）或红“马”（第 9 行第 5 列）隔着就能将军，这样红“相”和红“马”就被黑“炮”牵制，战斗力大减。如果程序可以认识到这种情况，对棋力的提高是很有帮助的。

对局面 p 下棋子间的牵制的分值量化表示如下，

$$e_4(p) = \sum \phi_i \times F(p) \tag{4.4}$$

式中，ϕ_i 表示牵制系数；$F(p)$ 为局面 p 中棋子间形成牵制的情况；$e_4(p)$ 表示局面 p 中单方总体牵制分值。其中，

$$F(p) = \sum f_n(M_i, M_j, M_k, ... | p) \tag{4.5}$$

$f_n(M_i, M_j, M_k, ... | p)$ 表示局面 p 中，棋子 M_i，M_j，M_k 等相互之间一个牵制多个或多个牵制多个的 n 种牵制情况的棋子牵制分值。例如图 4-8 例中的一个牵制就是，M_i 可表示位于第 7 行第 5 列的黑“炮”，M_j 和 M_k 分别表示位于第 5 列的红方的“相”和“马”。

同时，棋子是否处于对方棋子的威胁下，是否被本方棋子保护，决定了棋子的安全系数。受到威胁的棋子应该给予惩罚，受到保护的棋子应该给予奖励。如果棋子下一步将被吃掉，则应该给一个负的附加值，但如果周围有己方棋子对其进行了保护，也就是说，如果对方吃了这个棋子，将会损失更大，则应该给一个较大的附加值。

下式给出了局面 p 下棋子间保护的计算方法，

$$e_5(p) = \sum \varphi_i \times G(p) \tag{4.6}$$

中国知网 https://www.cnki.net

式中，φ_i表示保护系数；$G(p)$为局面p中棋子间保护情况；$e_5(p)$表示单方获得的总的保护分值。其中，

$$G(p)=\sum g_n(M_i,M_j,...|p) \tag{4.7}$$

$g_n(M_i,M_j,...|p)$表示在局面p中，棋子M_i对棋子M_j等的保护。

棋子间威胁分值的计算与计算保护值的方法相同，只是在计算单方总的威胁或保护分值时，如果是保护，就加上$g_n(M_i,M_j,...|p)$的值（即奖励），如果是威胁，就加上$-g_n(M_i,M_j,...|p)$的值（即惩罚）。

4.2.5 特殊棋型的评价

除了以上几个基本的评估模块外，评估函数还应考虑一些影响棋局发展的特殊棋型，例如连环马，担子炮，沉底炮，空头炮，相士的特殊棋型等，这些棋子之间的配合作战在本文的评估函数设计时也有所体现。例如，在程序设计中给定空头炮的威胁分值为：

HollowCannonThreat [10] = { 0, 0, 0, 37, 43, 55, 57, 67, 53, 49}

一般来说空头炮位置越高对对方的威胁越大，分值也就越大，根据经验在己方九宫中间正前方威力最大。

给定中炮的威胁分值为：

HorseCentralCannonThreat[10] = { 0, 0, 0, 43, 35, 24, 18, 9, 9, 9}

一般来说高度越低威胁越大，根据经验，可设炮镇窝心马的分值是该值的 4 倍。

给定沉底炮的威胁分值为：

BottomCannonThreat [9] = { 25, 21, 8, 0, 0, 0, 8, 21, 25}

一般来说越靠近边线威胁越大。

再有，“重炮”有可能形成杀棋，所以在评估的时候不能仅仅看作是后面的炮将军，应当特殊考虑。单独一个马的威力可能非常有限，可一旦形成“连环马”威力就大增，并且区分两个马是否能互相保护而给予不同的分值，能保护对方的马则设定较高的分值，否则给予较低的分值。这样连环马的分值设为：

InterlinkHorse[4][2]={{17, 17},{17, 9},{9, 17},{9, 9}}

InterlinkHorse 四组值的意义分别为：{17, 17}表示两个马互相能保护，{17, 9}表示仅前者能保护后者，{9, 17}仅后者能保护前者，{9, 9}双方都不能保护对方。

4.2.6 对战术的评价

中国象棋经过上千年的发展，形成了很多具有很强的攻击力和杀伤力的中局战术，比较典型的例如捉双战术、闪击战术、谋子弃子战术等等，将这些战术考略到中国象棋计算机博弈中，当对弈局面中出现这样的战术时，给予适当的附加值，也可以提高审局的准确性。

4.2.7 评估函数模型

中国知网 https://www.cnki.net

根据上面的介绍，一个局面单方的评估函数的计算就可以写成：

估值分数=子力值+位置值+灵活性+协调性+保护（威胁）+特殊棋型+战术

在数学上，评估函数的计算方法为：

$$\begin{aligned} E^R(p) &= \sum k_i^R e_i^R(p) \\ E^B(p) &= \sum k_i^B e_i^B(p) \end{aligned} \tag{4.8}$$

其中 p 为待评估的局面；$e_i^R(p)$ 和 $e_i^B(p)$ 为特征值函数，即上面介绍的各个评价模块；k_i^R 和 k_i^B 为特征系数，也就是每个模块所占的权重，并且 $\sum k_i^R = \sum k_i^B = 1$；$E^R(p)$ 和 $E^B(p)$ 分别为 p 局面下棋盘上红方和黑方的评估值。该公式表明，评估函数就是分别对当前棋盘的若干评价模块进行计算，获得其特征值，所有这些特征值与其特征系数乘积的累加，得到最终的评估函数值。显然，评估函数是一阶线性的。

按照式(4.8)分别计算红方和黑方的评估值，那么局面 p 总的评估值就是，

$$\begin{aligned} E &= E^R(p) - E^B(p) \\ &= \sum k_i^R e_i^R(p) - \sum k_i^B e_i^B(p) \end{aligned} \tag{4.9}$$

通过计算，如果 $E > 0$，则局面对红方有利；如果 $E < 0$，则局面对黑方有利。

本文在设计博弈系统的评估函数时，对上述几个评估模块都进行了考虑。但也不是所有博弈系统都必须包含这些模块，有的引擎舍弃了某个模块，或把几个模块融合起来考虑，有的引擎也会多考虑一些额外的信息，没有统一的规定，但不管怎样设计评估函数，都难免会存在缺陷，而在路径选择时产生波动，这是问题本身所固有的。但过大的知识量处理会令程序运行缓慢，而知识不足又会让程序犯战略性错误，在决定使用哪些评估信息时，怎样兼顾博弈性能和引擎速度是很关键的前提。

4.3 估值参数优化

我们知道，要使计算机每次都作出正确的选择，一个很重要的前提就是需要有准确无误的评估函数，然而实际上设计这样的评估函数是不可能的，假如可能，就不需要搜索算法了，而直接用评估函数取其着法为最大值的那一步就可以了。假设有一个评估函数是准确客观的反映局势的，称这个估值为理想评估函数，我们只可能逼近这个这个理想评估函数，而不可能到达，这是问题的固有难度所决定的[29]。

由 4.2 节的讨论可知，在建立评估函数时需要考虑很多因素，包含很多评估模块，而每个模块又涉及到很多参数，模块之间又会有很多反映哪个模块作用大，哪个模块作用小的权重系数，这样建立起来的评估函数即使最简单的都要包含上百个参数，高水平的评估函数包含的参数甚至可以达到几千个，“深蓝”的参数就有四千多个，这就给参数的优化带来很大困难。例如一个简单的评估如式(4.10)所示，

$$\begin{aligned} E^R &= k_1^R e_1^R + k_2^R e_2^R + k_3^R e_3^R + \cdots + k_i^R e_i^R + \cdots, \quad i = 1,2,3,\cdots,n \\ E^B &= k_1^B e_1^B + k_2^B e_2^B + k_3^B e_3^B + \cdots + k_i^B e_i^B + \cdots, \quad i = 1,2,3,\cdots,n \\ E &= E^R - E^B \end{aligned} \tag{4.10}$$

中国知网 https://www.cnki.net

其中E^R表示某局面红方的评估值，E^B表示相同的局面下黑方的评估值，e_i^R和e_i^B表示评估模块，k_i^R和k_i^B表示各模块间的权重系数，E表示局面的评估值。通过参数调整使局面评估值E更准确的反映局面的真实值，除了要调整模块内部的参数，模块间的权重系数也需要调整。如何对这些参数进行调整优化以使其更接近理想评估函数，是目前主要研究的对象。

4.3.1 手工调整

目前最常使用的优化估值参数的方法是利用大量的测试局面进行手工调整[12]，这也是本文在优化评估函数时主要使用的手段。这种方法是利用人类的象棋经验知识来调整参数值，比如，从经验上可以知道，一个车的价值要比一个兵大，给车赋予比兵大的数值，马炮则赋予位于其间的值；马和炮的地位相当，给予它们相当的数值，以避免盲目换子；如果对弈中使用了一些优秀的战术配合，那么就给予一定数值的奖励，等等。将这些经验放进评估函数中反复对弈，然后不断修正参数，找出一组性能较高的参数。

实际上，4.2一节中构建评估函数的过程就是用人工手段进行参数调整的，本文在设计评估函数时，用手工方法找到合适的参数主要使用了以下手段[32]：

(1) 规格化(Normalize)

如果只是关心评价的顺序，而不怎么关心评价值，那么可以把每一项都乘以同样的常数。这就意味着，对某个特定的模块，例如兵的价值，可以硬性设一个值，其他值就可以表示成它们相当于多少个兵。这个做法可以减少一个需要设定的参数。

(2) 约束法(Deduce Constraints)

通过考虑希望电脑作出怎样的判断，就可以确定一些参数。例如在对弈中即使赚到一个兵，用车换象或马还是坏的，但如果能赚到两个兵那还是好的，因为在考虑子力价值是要防止换单兵，鼓励换双兵。这样的条件越多，合适的权重组合就越少。在开始设定权重值的时候，这个方法通常可以得到合适的值，但是在后面仍然需要做一些调整。

(3) 交手法(Hand Tweaking)

这是调整评估函数时非常常用的方法，通过让程序对弈，来找到它的优势和弱点，猜测哪些参数会让程序更好，然后挑选新的参数。这个方法可以很快得到合理的结果，但是需要对棋类知识有足够的了解，便于根据程序的对局来做分析，从而知道程序的问题在哪里。

手工调整是象棋引擎调整估值参数的主要手段之一，把人类的知识和经验尽量准确客观地“教授”给计算机，是提高棋力的普遍做法，虽然比较费时，但是很有效。通过不断的试验、修改参数值，可以得到不错的结果。但是人类的知识毕竟具有一定的局限性，评估函数也不能包含所有情况，参数之间往往又互相联系，调整某个参数可能解决了某类问题，但又可能给其它问题的解决带来麻烦，在这种情况下很难实现全局下的最优组合。

4.3.2 机器自学习

中国知网 https://www.cnki.net

手工调整相当于把人类经验赋予计算机，“教”计算机如何下棋，而利用人工智能算法可以根据大量的对局进行参数自动优化，即通过参数自动优化使局面评估值E更接近理想评估函数值。下面介绍几种通过机器自学习优化参数的方法。

(1) 爬山法(Hill-Climbing)

爬山法[33]通过对参数进行小范围的试探来寻找最优解，一次只能对参数作一点小改变，然后测试博弈程序的性能是否提高了，仅当性能提高时才采纳这个改变，需要重复很多次。这种方法很慢，并且受初始采样值的限制，很容易陷入局部最优，即评价可能很差，但是任何很小的改变都会使评价更差。

(2) 模拟退火(Simulated Annealing)

模拟退火[33-34]是一种基于蒙特卡罗(Monte Carlo)算法的启发式随机搜索算法，它没有蒙特卡罗算法多次寻优的过程，也不像爬山法那样依赖初值。在优化参数时，类似于爬山法，模拟退火法也是对权重做改变来提高成绩，如果所做的改变没有提高成绩，也会根据一个随机的几率采纳这个改变，试图跳出局部最优。这个方法需要给定一些几率，从几率高、梯度大的条件开始，然后逐渐减小。模拟退火法比爬山法更慢，但是最终可能得到比较好的值。

(3) 遗传算法(Genetic Algorithm, GA)

遗传算法[33, 35-36]是建立在自然选择和自然遗传学机理基础上的迭代自适应概率性全局优化算法，因其模仿生物的遗传机制而得名，最早由美国密执安大学 J.H.Holland 于 1975 年提出，它通过染色体的复制，交叉，变异等操作，实现个体适应性的提高。

遗传算法可以同时维护多组较好的参数，通过向其中添加一组新参数，通常是将从几组老参数中选出的值组合在一起作微小的变化，然后同几组老的参数比较孰优孰劣，将最坏的一组去除。遗传算法是从几组参数开始，而不是一组参数，具有很好的全局搜索能力，搜索速度也很快，而且此算法能将搜索重点集中于性能高的部分，能较快地求出最佳参数，鲁棒性也明显优于前两种算法，所以在计算机博弈中最有可能取得成功。

图 4-9 表示了使用遗传算法优化估值参数的过程。

中国知网 https://www.cnki.net

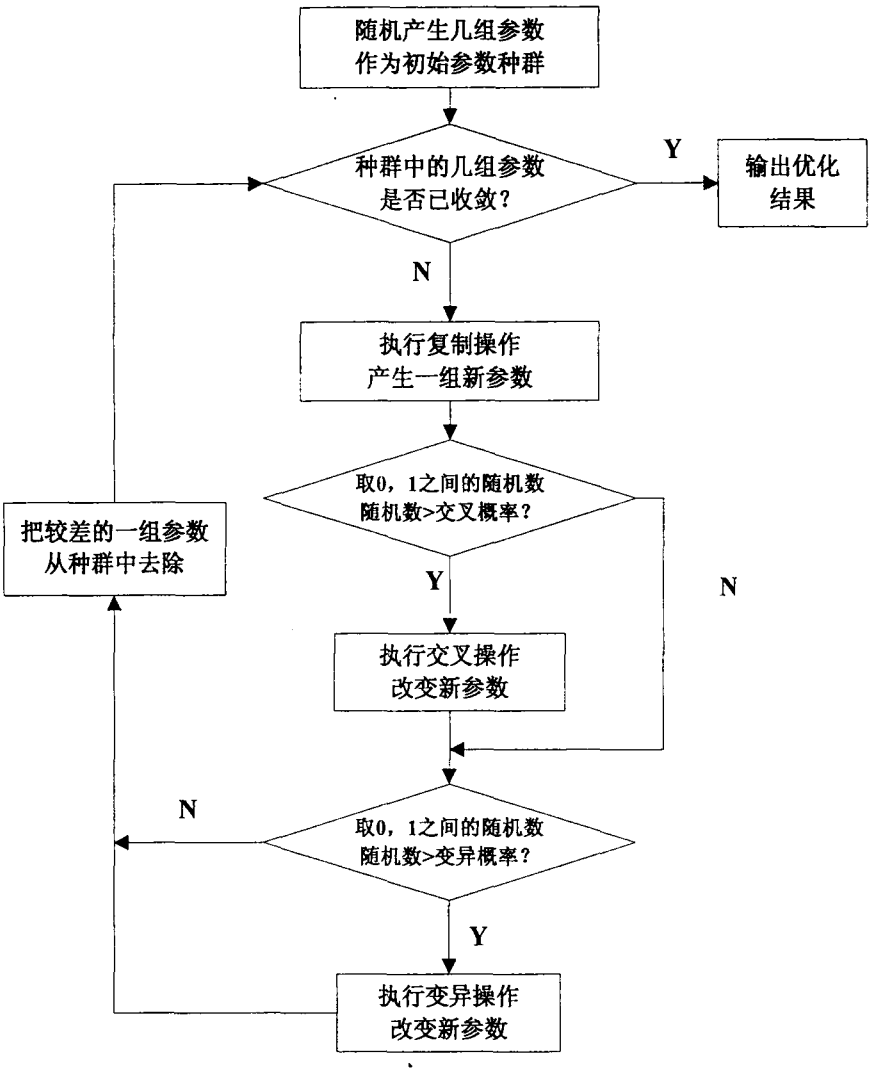


图 4-9 遗传算法优化估值参数的流程图

Figure 4-9 The flow chart of optimizing parameters using GA

在使用遗传算法优化参数的过程中要注意以下几个问题：

种群数目：这是影响算法性能的重要因素，种群数目过大会导致运算时间过长，结果收敛缓慢，而种群数目太小则可能导致算法精度过低，甚至可能失败而得不到最优解。

交叉概率：用于控制交叉操作的频度。此概率过大会导致种群中的个体更新过快，较优的个体不能稳定的保持和传递信息，而概率过小则会使优化的过程过于缓慢甚至停止不前。

变异概率：一般使用一个很小的概率值，用于控制产生变异个体的频度，防止整个种群中任意参数的几个基因保持不变。概率太大遗传信息不能稳定传递，会使优化过程变成随机过程，失去确切的方向而导致失败。

目前已经有象棋引擎把遗传算法应用到评估函数[38]，通过试验得出的结果可以看出遗传算法确实有效，但需要遗传很多代，或者有一个好的权重组合作为祖先，但事实上，调整出的评估函数值仍然比手工调整的差。

(4) 神经网络(Neural Networks, NN)

神经网络[35-37]是 20 世纪 40 年代由心理学家 W.S.McCulloch 和数学家 W.Pitts 提出的

中国知网 https://www.cnki.net

一种模拟人脑神经元活动的技术。神经网络是一个并行和分布式的信息处理网络结构，一般由许多个神经元组成，每个神经元只有一个输出，可以连接到很多其他的神经元，每个神经元输入有多个连接通道，每个连接通道对应于一个连接权系数。图 4-10 是一个典型的前向神经网络结构图。

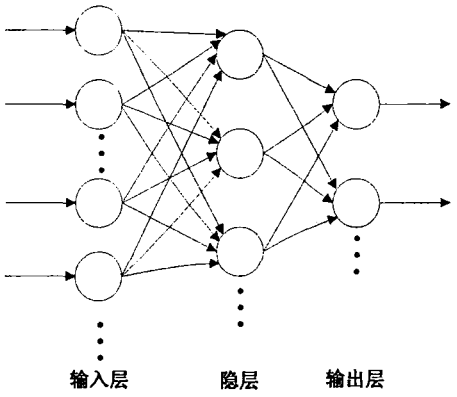


图 4-10 前向神经网络结构图

Figure 4-10 The fig. of feed-forward NN structure

它包含输入层、隐层和输出层，隐层可以为一层或多层，每层上的神经元称为节点或单元。输入单元的状态代表输入此网络的原始信息，每个隐层单元的状态取决于输入单元的状态和该输入单元与隐层元之间的连接权值。同样，输出单元的行为取决于隐层元的状态和隐层元与输出单元之间的权值。

将神经网络应用到评估函数中，实际上更多的是一种评估函数的类型，而不是用来选择权重的：神经元是阈值（输入权重的和）的函数，第一层神经元输入关于局面的性质（例如位棋盘表示中的某几个位）就可以构造一个网络了，然后把前一层的结果输入到后一层。因此实际上单输入神经元的单层网络就等同于上面讨论的一阶线性评估函数。通过确定网络何时会作出坏的评价，让每个权重增加或减小看是否会评价的更好，有点类似于爬山法。

跟其它自学习的方法相比，神经网络的好处在于，它不需要很多的棋类知识就可以让程序有个比较好的评估函数。但是尽管这样，就目前研究的情况来讲，根据我们的象棋经验知识来做评估函数，要比机器自学习做得好，并且做得快。

以上简要介绍了几种可应用到计算机博弈评估函数中的智能算法，但究竟采用何种智能算法才能带来良好的结果，即使国际象棋也没有非常成功的先例。目前计算机博弈研究界还没有一种很好的参数学习修正的方法，把机器自学习应用到参数优化中的研究实验显示，大部分通过自学习得出的结果比手工调整还差，有待进一步研究和尝试。

4.4 参数优化策略

4.4.1 藐视因子

根据局面评估函数的数学表达式：

$$E = E^R(p) - E^B(p) \tag{4.11}$$

中国知网 https://www.cnki.net

其中$E^R(p)$是红方的评估值，$E^B(p)$是黑方的评估值，E是局面p总的评估值。定义棋局的输赢是很容易的事：如果将死对手只需要E返回一个很大的正数，如果被对手将死E就返回一个很大的负数。但是在和局时返回$E=0$并不好：如果程序对弈比它强的棋手，那么一个和局可能对程序来说非常不错，但是当程序对弈一个很弱的棋手时，和局却很失败。

在和局时加入一个“藐视因子”（Contempt Factor）[32]，使程序根据对和局的渴望程度来调整和局分值。下面的公式表示加入藐视因子ξ后和局评估值的计算，

$$E_{和局}=\xi\cdot\left[E^R(p)-E^B(p)\right] \tag{4.12}$$

由式(4.11)可以看出，当局面稍差（规定电脑执黑，相对电脑而言），即黑方的评估值稍小于红方的评估值时，E为一个正数。在这种情况下，给式(4.12)中的藐视因子ξ一个稍负一点的数，则E就变为负数，此时程序也会试图取胜。

典型的藐视因子就是一个稍负的数。根据对弈双方的情况选择不同的藐视分值在策略上是可行的。如果在很激烈的中局并且计算机稍微落后，通常可以考虑给ξ一个绝对值大些的负数，这种条件下电脑就可以为胜利而战。一般认为，在开局阶段合理的藐视因子是-0.50个兵，整体上合理的藐视因子是-0.25个兵，而在残局中藐视因子取零就比较合理，或者用一个绝对值很小的负值就可以了。

在本文的评估函数设计中，当局面的评估值接近零的时候，如式(4.12)所示加入藐视因子，并通过实验最终给定其值为-30。经过大量测试发现，加入藐视因子后确实能使电脑在接近和局时，选择有利于其取胜的路径走棋，在一定程度上减少了失误，提高了获胜率。

4.4.2 局势因子

4.2节所讲的评估函数的构建实际上是用静态估值方法给局面打分，评估局面的过程。所谓静态估值方法就是针对一定深处的节点，通常是搜索到的最底层的节点，给这些节点打分，然后根据其值大小搜索选择合理的走法路径。这种方法存在以下问题[4]：

(1) 假定评估函数是绝对可靠的，每次都是以该估值为依据。实际上，肯定是有误差的，否则就不叫评估函数了。而且一旦在某个下级节点处发生误差，这种误差沿着通路向上传播，将有可能造成上级节点作决策时产生关键性的误差，并且这种失误是无法纠正的。

(2) 在计算估值时，博弈者在整个对弈过程中，所有局势均使用同一个评估函数。而在实战中，由于棋局的变化，在不同的局势下，即使是相同的棋面，对其进行评估的结果也可能是不同的。局势越跌宕起伏，评估结果偏差越大，这种误差延着回溯通道向上传播，将严重影响博弈者的决策。

(3) 博弈树搜索算法的静态估值是基于双方棋力差，每次都以该估值的绝对量为依据。而在实战中，双方棋力的消长是以各自在不同棋面下占有总棋力百分比为标志的，静态估值仅反映双方棋力的强弱，没有反映出差值量的权重。

(4) 电脑对弈的每一步都是先采用向下展开若干步，再回溯搜索确定博弈走法，是一

中国知网 https://www.cnki.net

种“由下而上”的方法。而在实战中，对于不同的局势每个棋手都会施加主观战略意图，伴随着一种“自上而下”的方法，思维方式的灵活变化使原有的静态估值方法无法体现博弈者的战略意图。

显然，如果能够克服静态估值的这些不足，在博弈中不断地根据对弈局势，调整己方的估值函数，使得己方对局势的估计优于对方或者和对方的估值方法尽可能的一致，就可以掌握博弈的主动权。

一种有效的估值方法就是在静态估值方法的基础上，充分考虑局势的变化和己方的博弈特点，引入一个合适的“局势因子”嵌入博弈双方的战略意图，适时地调整评估函数，以提高实用性，这种方法又称为二次估值法[40]。据此，数学表达式(4.11)就可以写为：

$$E = \alpha \cdot E^R(p) - (1-\alpha) \cdot E^B(p) \tag{4.13}$$

另设 $RedVl = E^R(p)$, $BlackVl = E^B(p)$，则上式写为：

$$E = \alpha \cdot RedVl - (1-\alpha) \cdot BlackVl \tag{4.14}$$

其中 E 表示棋面上棋子的总评估值，$RedVl$ 和 $BlackVl$ 分别代表一个局面上红方棋子的价值总和和黑方棋子的价值总和，而 α 和 $(1-\alpha)$ 分别代表红棋与黑棋价值在总的估值中占有的权值比例，即局势因子，并且 α 满足下面的条件：

$$0 < \alpha < 1 \tag{4.15}$$

局势因子的大小是博弈者战略意图的反映，不同的局势因子的选择将导致对局势不同的判断，在象棋中为当己方意图采取攻势的情况下，己方的局势因子应该大于对方的局势因子（即己方的局势因子应大于 0.5）；当己方意图采取守势的情况下，己方的局势因子应该小于对方的局势因子（即己方的局势因子应小与 0.5）；当己方和对方势均力敌时，己方选择的局势因子应该和对方等同（即己方的局势因子应等于 0.5）。即使对于某个确定的局面，并且具有确定的估值，选择不同的局势因子也会导致不同的博弈路径及其结果。

局势因子的确定与选择：

定义：

$$\Delta_{ij} = \frac{RedVl_{ij}}{RedVl_{ij} + BlackVl_{ij}} \quad (i = 1,2,\cdots,n; j = 1,2,\cdots m) \tag{4.16}$$

Δ_{ij} 为局势 i 中第 j 个局面下的红方棋力所占比例。其中 $RedVl_{ij}$ 和 $BlackVl_{ij}$ 分别为红方和黑方在局势 i 中第 j 个局面下的棋力值。则有：

当 $0 < \Delta_{ij} < 0.5$ 时，定义红方局势为守势；当 $0.5 < \Delta_{ij} < 1$ 时，定义红方局势为攻势；当 $\Delta_{ij} = 0.5$ 时，定义红方局势为和势。另设，

$$c_{ij} = RedVl_{ij} + BlackVl_{ij} \tag{4.17}$$

中国知网 https://www.cnki.net

为局势i中第j个局面红方和黑方评估值的和，c_{ij}和Δ_{ij}为局势i的数值表示，其中：

$$f(\Delta_{ij}, c_{ij}) = \Delta_1 \tag{4.18}$$

局势变化满足下面的条件：

$$\left|\Delta_j - \Delta_i\right| \geq \Delta_2 \tag{4.19}$$

其中，Δ_j，Δ_i分别为局势i和局势j下红方棋力所占的比例。称Δ_1为局势阈值，Δ_2为局势变化阈值。

局势因子的大小是衡量双方攻守形式和策略的一个度量，通过局势因子能够较好的对局势作出正确的评估。设α_i为局势i时体现攻守策略的局势因子，令棋面局势为1，则局势因子应满足以下条件：

$$0 < \alpha_i < 1 \tag{4.20}$$

在对弈中，对局势因子的选择通常有两个方法：

1、先验法

根据以往对弈经验和对博弈对象的了解，事先确定己方的博弈策略，确定一个合适的局势因子嵌入到估值函数中，并在对弈中适时调整。例如，己方在以往对弈中若常常采用守势开局，则选择$\alpha_i \in (0, 0.5)$中适当的值；若己方在以往的对弈中常常采用攻势开局，则选择$\alpha_i \in (0.5, 1)$中适当的值。

先验法的好处在于，能较好地掌握对弈的主动权，引导局势向有利于己方的方向发展。

2、差分修补法

在实际对弈过程中应该调整局势因子使得己方棋力在总棋力中占的比例总是尽可能大。对弈双方采用的博弈策略往往不同，对相同局面的评估也不一样，存在路径选择的差异，比较Δ_{in}和Δ_{ix}，若

1) $\Delta_{ix} > \Delta_{in}$，则需要调整局势因子，若开始为均势即$\Delta_{in} = 0.5$，根据式(4.14)则

$$E_{in} = 0.5(RedVl_{in} - BlackVl_{in}) \tag{4.21}$$

$$E_{ix} = \alpha \cdot RedVl_{ix} - (1-\alpha) \cdot BlackVl_{ix} \tag{4.22}$$

由$E_{ix} \geq E_{in}$得，

$$\alpha \geq \frac{0.5 \times (RedVl_{in} - BlackVl_{in}) + BlackVl_{ix}}{RedVl_{ix} + BlackVl_{ix}} \tag{4.23}$$

根据上式并结合己方策略选择一个合适的局势因子α。

2) $\Delta_{ix} < \Delta_{in}$，除非己方策略需要，不需要调整原有的局势因子。

在同一个局势下，由于博弈者出发点和对局势的理解不同，对于局势因子可能有多种选择，为了能达到全局的最优判断，还应该对局势因子进行优化，可以使用遗传算法等人工智能算法求解局势因子[41]，以达到选择全局最优博弈路径的目的。

中国知网 https://www.cnki.net

使用局势因子，建立二次估值函数（式(4.14)）评估局面的步骤：

(1) 按照静态估值方法，首先对棋面进行初步评估（式(4.9)）。通过对博弈双方各自棋子的评估值比较，判断己方的攻守形势。

(2) 在第一个回合中，选取适当的局势因子(若初始棋局为均势，取局势因子0.5)，得出不同走法下局面的估值后，按传统方法搜索博弈路径。结束后，记录博弈路径上红棋棋力在棋局中所占总棋力的百分比（式(4.16)），给出博弈走法。

(3) 在第二个回合中，先使用原局势因子下的二次评估函数对棋面进行估值和搜索，对博弈路径上红棋棋力在总棋力中所占百分比与上回合中记录的棋局中红棋棋力的百分比进行比较。

(4) 若差值大于设定的局势变化阈值Δ_2，则使用新局势因子下的二次评估函数对棋面进行估值和搜索，确定新的博弈路径。结束后，记下博弈路径上新的红棋棋力在棋局中总棋力的百分比，给出博弈走法。

(5) 若差值不大于设定的局势变化阈值Δ_2，则记下博弈路径上新的红棋棋力在总棋力中所占百分比，给出博弈走法。

(6) 以此类推，直至终局。

下图是二次估值算法应用于评估函数的流程图。

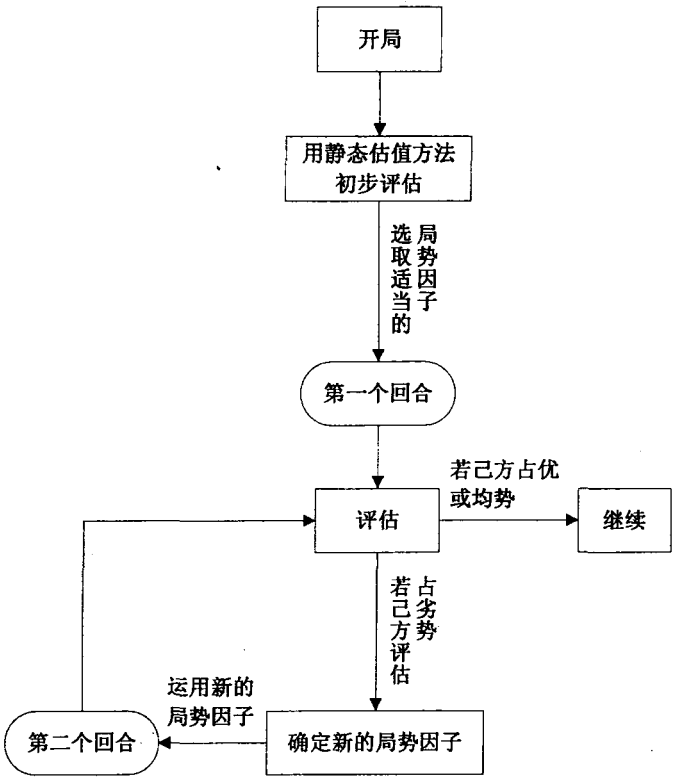


图 4-11 二次估值的评估流程图

Figure 4-11 The flow chart of secondary evaluation

本文在优化评估函数时尝试了在原有评估函数的基础上加入“局势因子”，通过对路径进行二次估值调整，在一定程度上提高了评估函数的实用性，有效提高了作战获胜率。

中国知网 https://www.cnki.net

但由于时间有限，未能对其进行更多的测试，在今后研究工作中有待进一步优化调整，以获得更好的成绩。

4.5 全动态调整估值参数

本节提出一种全动态调整估值参数的方法，即全动态分配棋子分数的思想。这种方法称之为“全动态”只是针对给局面打分采用全动态分配棋子价值而言，实际上仍是静态估值方法，即只对搜索的最底层节点打分。以棋子的灵活度值为例，对此方法进行介绍。

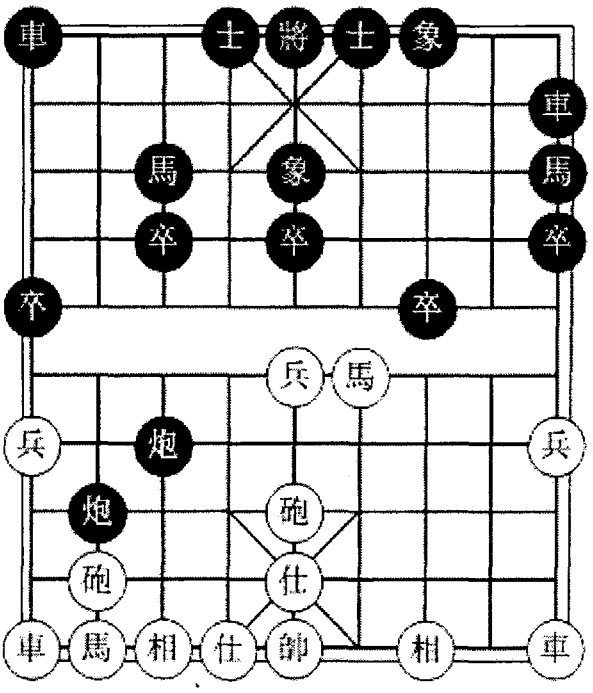


图 4-12 对弈局面 1

Figure 4-12 Game board 1

对于静态评估来说，根据经验分配棋子的灵活度系数如表 4-2 所示。而对于动态分配棋子分数来说，一开始我们并不知道哪个棋子灵活度系数大一些哪个小一些，而是赋予每个棋子相等的灵活度系数，假设都为 10，这样在某个对弈局面（以图 4-12 为例），左下角红车可走的位置是 2，那么根据公式(4.3)，它的灵活度值就是10×2为 20，同时它对旁边的红马有保护作用，奖 10 分，即总分为 30 分；位于九宫里的红炮可走的位置是 8，其中包括一个吃子位置，奖 10 分，则红炮的灵活度值为$10\times8+10=90$，显然这样一个红炮比红车攻击力要大得多，从图 4-12 中的局势我们也可以看出。但是如果采用静态评估，根据表 4-2 给出的灵活度系数，那么红车的灵活度值为$8\times2=16$，红炮的灵活度为$2\times8=16$，红炮的攻击力与红车等价，显然这与事实不符。并且，利用全动态评估方法考虑更多的因素，能真实客观地反映棋局的好坏。

全动态调整估值参数的思想：初始认为所有棋子的价值是相同的，不分强弱，在对弈中让电脑自己“体会”哪个棋子更有价值，根据某些因素给予各个棋子不同的加分，最终各个棋子具有不同的分值，体现棋子对棋局影响的不同。全动态评估参数其实是使参数

中国知网 https://www.cnki.net

自动寻优的过程。

建立全动态评估棋子价值的模型，如下式所示：

$$e(p)=\sum \mu_i(B_i \times T_i(p)) \tag{4.24}$$

其中 B_i 是设定的棋子在各个特征模块中的初始值，$T_i(p)$ 是局面 p 下棋子对应的特征系数，表现为可走着法数，吃子着法数，保护几个棋子，受到几个棋子威胁，与其它棋子牵制、协同作战的情况，距将（帅）的最短距离等等；μ_i 为特征模块间的权重；$e(p)$ 为局面 p 下单个棋子总的价值。

根据全动态估值的思想，每个棋子的在同一个模块里的初始值是等同的，例如，在棋子灵活性的计算中，各个棋子的初始值可都表示为 B_1，而计算吃子走法值时，各个棋子的初始值都设为 B_2，依次类推。这样，通过式(4.21)分别计算各个棋子在同一局面下总的评估值，然后红子和红子求和，黑子和黑子求和，最后红方与黑方作差就得到该局面的评估值了。

将式(4.24)展开，即

$$e(p)=\mu_1 B_1 T_1(p)+\mu_2 B_2 T_2(p)+\mu_3 B_3 T_3(p)+\mu_4 B_4 T_4(p)+\cdots \tag{4.25}$$

由上式可以看出，全动态评估思想就是，计算局面中每个棋子的评估值大小 $e(p)$，通过比较各个棋子评估值的大小，来反映棋子对局势的影响力。评估值大的棋子对局势的影响力大，反之，评估值小的棋子对局势的影响力就小。

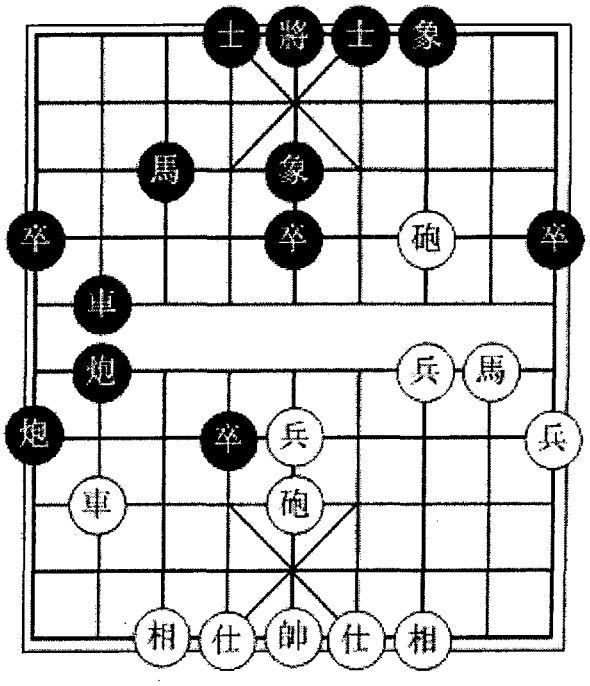


图 4-13 对弈局面 2

Figure 4-13 Game board 2

对图 4-13 进行全动态估值，以黑卒（第 7 行第 4 列）、黑马（第 3 行第 3 列）、黑车（第 5 行第 2 列）为例。设棋子的灵活度基数 $B_1=10$，棋子间的威胁保护基数为 $B_2=15$，

中国知网 https://www.cnki.net

那么黑卒的评估值是这样计算的：它可走的位置为 2，还有一个吃子位置，一共是 3，加入吃掉红兵后，红兵的评估值也会加给黑兵，红兵的评估值为 10（只有一个可走位置）。则利用(4.25)式，在不考虑权重的情况下（下同），

$$e_{B_PAWN} = B_1 \times 3 + 10 = 10 \times 3 + 10 = 40$$

黑马的评估值计算：它有 4 个可走位置，对两个黑卒（第 4 行第 1 列和第 4 行第 5 列）都有保护作用（防止红炮吃卒），所以它的评估值为：

$$e_{B_HORSE} = B_1 \times 4 + B_2 \times 2 = 10 \times 4 + 15 \times 2 = 70$$

黑车的评估值计算：它有 12 个可走位置，对炮（第 6 行第 2 列）有保护作用（防止第 8 行第 2 列的红车与该炮兑子），它的评估值为：

$$e_{B_CAR} = B_1 \times 12 + B_2 \times 1 = 10 \times 12 + 15 \times 1 = 135$$

用相同的方法计算黑方各个棋子的评估值，然后求和就可以得到黑方在这个局面下的总评估值 $E^B(p)$。

同理，计算红方的评估值 $E^R(p)$。如果轮到红方走棋，那么局面的评估值就是：

$$E = E^R(p) - E^B(p) = \sum e_i^R(p) - \sum e_i^B(p) \tag{4.26}$$

该方法只是作为一种思想提出，而没有实现，但通过对一些局面评估值的计算，利用全动态评估方法能真实客观地反映棋局的好坏，对计算机博弈而言也是符合客观实际的。

4.6 评估函数优劣的测试

在建立评估函数和对估值参数优化的过程中，每作一次修改都要对其进行实验测试，看系统性能是否提高了。本文在对评估函数的优劣进行测试时使用的方法有：

(1) 对一批人类棋手下出的棋局进行测试，看看其做出正确选择的几率是多少。

(2) 同其他博弈程序对弈，看使用某一组参数时取胜的几率有多大。

(3) 让使用两组不同参数的该程序自相对弈，看看使用哪组参数取胜的几率更大。这种方法存在一些风险，程序必须具备一定的随机因素，否则多次对弈的过程可能完全一样，而无法得知获胜几率。

(4) 将使用某一组参数的评估函数同向下搜索几层的结果作比较，其结果越相似，说明其准确率越高。这种方法使用起来比较容易，但也有滑向失败的可能，例如，当估值取一个常数时，无论多少层估值都是相同的，但这时的参数绝不是最优的。

通过大量的实验测试和不断调整，本文的评估函数可以较好的反映局面优劣，并且在加入藐视因子和局势因子后，与之前的系统进行对弈获胜率显著提高了。评估函数每做一次修改都要进行大量测试，工作量非常大，是整个博弈系统设计中最复杂的工作。

4.7 小结

本章主要对评估函数进行了研究，包括评估函数和博弈性能的关系，评估函数的构成

中国知网 https://www.cnki.net

和数学模型，并主要研究了估值参数的优化方法和策略。实际上，局面的评价就是知识（评估函数的构成因素）加上优化（对评估函数里的参数进行调整）的表示。评估函数是整个象棋引擎里很特别的一部分，具有极为广泛的内容。搜索技术已经有比较成熟的的算法，着法生成可以根据行棋规则再加上一些辅助着法生成的方法来处理，而对于局面的评估则需要进行很深入地分析。

本文在设计系统的评估函数时，对固定子力值、棋子的位置值、灵活度、棋子间的牵制、威胁与保护，以及一些特殊棋型等都给予了考虑，并在评估过程中加入了藐视因子和局势因子来调整博弈路径的选择，通过大量测试和不断地优化调整，为系统建立了一个较好的评估机制。

但由于时间有限，加上评估函数问题本身的固有难度，对这部分的设计还需要不断地完善。例如在本系统评估函数的构建中，虽然比较全面的考虑了影响局势的主要因素，但必然还存在一些没有认识到的盲点，比如是否需要增加更多的对提高棋力有益的象棋知识和经验到程序里，应采用怎样的权重加入等等。在优化方面，尽管已有一些博弈引擎在小的局部使用了机器自学习的方法来调整评估函数的部分参数，但这也只是作为尝试，效果并不比手工调整更理想。这主要是因为像象棋这种复杂的对象，其规律由很多特征组成，各种特征交错作用，同时有些重要特征出现的几率并不高，使得大部分参数的学习效果并不理想，工作量也非常大。所以手工调整仍然是最快速有效的优化评估参数的手段。

总之，无论是在评估函数构建方面，还是在优化方面，如何提高局面评估值的正确性，即如何更接近理想评估函数是一项艰巨而复杂的工作，需要更深入地研究试验，而设计高水平的引擎需要程序员掌握大量的象棋知识，甚至需要请象棋大师加入进来参与调整。

中国知网 https://www.cnki.net

5 博弈系统开发与实现

5.1 系统程序的主要组件

本文的博弈系统程序是在 VC++环境下实现的，主要组件包括：数据结构，博弈树搜索算法，评估函数，以及一个用户界面。图 5-1 是本文计算机象棋博弈系统的流程图，它表现了一个完整的博弈系统的执行过程。

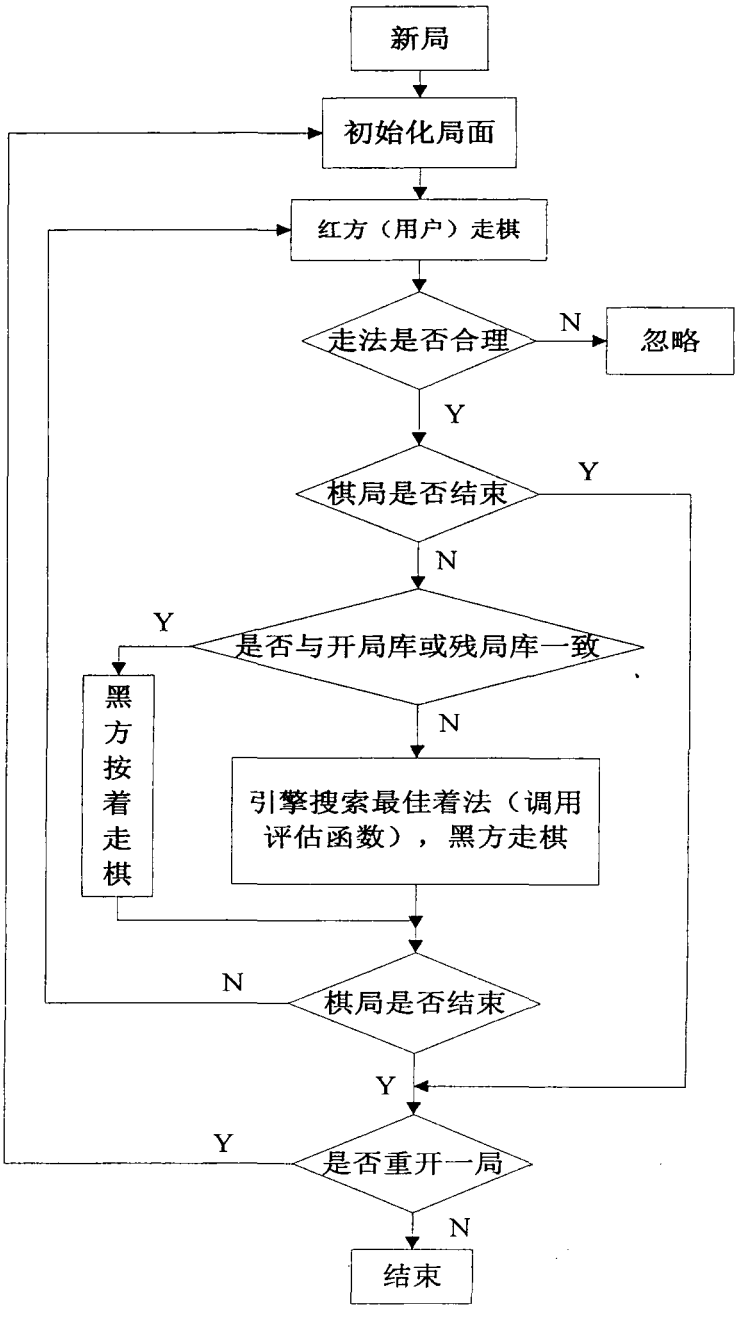


图 5-1 中国象棋计算机博弈程序流程图

Figure 5-1 The flow chart of Computer Chinese Chess program

系统设计的主要部分包括：

(1) 结构 ChessMove 和 ChessUnMakemove：定义了系统中最基本的数据结构。

中国知网 https://www.cnki.net

(2) 类 CMoveGenerator：系统的着法生成器。它记录了中国象棋的行棋规则，是对计算机在搜索过程中产生的着法的实现，以及判断人类棋手走棋是否合理的依据。

(3) 类 CSearchEngine：搜索引擎的接口类，任何搜索算法都是它的子类。

(4) 类 CHistoryHeuristic 和类 CTranspositionTable：两种搜索的优化算法，单独成类，对搜索算法进行优化。

(5) 类 CAlphaBeta_HH_TT：搜索引擎的具体实现，在 a-ß 剪枝搜索的基础上，用历史启发、置换表和静态启发算法进行优化，并加入了空着裁剪作为增强手段。

(6) 类 CEveluation：评估函数的具体实现，负责估值方法的最终生成，包含本文设计的各个估值模块以及估值策略。

(7) 类 CChessDlg：用户界面的实现，包括博弈系统软件所需的一些功能控件和接受用户操作的输入。

下面详细介绍数据结构和评估函数的实现。

5.1.1 数据结构

按第二章介绍的编码方法，本系统使用 0~89 的一维数组 Position[90]表示棋盘坐标，分别用-7~+7 和 0~32 表示棋子类型和棋子个体，且-1~ -7 表示黑方棋子，+1~+7 表示红方棋子，1~16 表示黑方棋子个体，17~32 表示红方棋子个体，0 表示无棋子状态。定义数组：

```
char ChessBoardIndex[90]; //棋盘索引数组
char ChessmanIndex[33];   //棋子索引数组
char MappingChess[90];    //映射数组
```

三个数组的初始化由图 2-2、图 2-3 和图 2-4 所示。定义如下判断（x, y 表示棋子种类编号）：

```
IsBlack(x)   (x<0); //判断棋子是否是黑子
IsRed(x)     (x>0); //判断棋子是否是红子
IsSameSide(x,y) ((IsBlack(x)&IsBlack(y))||(IsRed(x)&IsRed(y))); //判断是否是己方棋子
```

棋子在棋盘中的位置和生成着法的结构是最基本是数据结构，本系统定义如下结构：

```
struct _chessmove
{
    short    ChessID; //是什么棋子
    char     From;    //着法的起始位置
    char     To;      //着法的目标位置
    int      Score;   //棋子的评估值
}ChessMove;
struct _chessUnmakemove   //主要用于悔棋
{
    char nchessID;   //棋子位置坐标
```


中国知网 https://www.cnki.net

```
    char mappingID; //映射数组坐标
}ChessUnMakemove;
```

在着法生成中，着法生成器 CMoveGenerator 类包含了中国象棋的行棋规则和走法生成两个部分。类的定义代码片段如下：

```
class CMoveGenerator
{
 public:
    static BOOL IsChecked(char position[90], int Turnside); //判断是否将军
    CMoveGenerator();
    virtual ~CMoveGenerator();
    static BOOL IsValidMove(int nFromX, int nToX); //着法是否有效
    short CreateCapMove(int nPly, int nSide);    //产生吃子着法，只为静态搜索服务
    short CreatePossibleMove(int nPly, int nSide); //产生所有着法
    ChessMove m_AllMoveList[10][80];      //所有着法的队列
    ChessMove m_CapMoveList[10][80];    //吃子着法的队列
    ChessMove m_NoCapMoveList[10][80]; //非吃子着法的队列
    ChessMove m_quiesceMoveList[20][30]; //静态搜索吃子着法
    short quiesceCount;      //静态搜索，产生的着法数
protected:
  //产生各棋子的吃子着法
    void Gen_KingCapMove(int i, int nPly);
    void Gen_CanonCapMove(int i, int nPly);
    void Gen_BPawnCapMove(int i, int nPly);
    void Gen_RPawnCapMove(int i, int nPly);
    void Gen_CarCapMove(int i, int nPly);
    void Gen_HorseCapMove(int i, int nPly);
    void Gen_ElephantCapMove(int i, int nPly);
    void Gen_BishopCapMove(int i, int nPly);
  //产生各棋子的所有着法
    void Gen_KingMove(int i, int nPly);
    void Gen_BishopMove(int i, int nPly);
    void Gen_ElephantMove(int i, int nPly);
    void Gen_HorseMove(int i, int nPly);
    void Gen_CarMove(int i, int nPly);
    void Gen_RPawnMove(int i, int nPly);
```


中国知网 https://www.cnki.net

```
    void Gen_BPawnMove(int i, int nPly);
    void Gen_CanonMove(int i, int nPly);
  ……
  };
```

类 CMoveGenerator 包含 2 个重要成员函数：IsValidMove()和 CreatePossibleMove()。IsValidMove()用来判断局面上的一个走法是否合法；CreatePossibleMove()用来枚举给定局面上的所有走法。

为了加速着法生成，本系统使用了模板比配和路向行向比特向量辅助着法生成，模板匹配定义了如下数组：

```
const char BishopMove[90][5]     //士（仕）的着法模板匹配数组
const char ElephantMove[90][5]   //象（相）的着法模板匹配数组
const char ElephantEye[90][5]    //象眼模板匹配数组
const char KingMove[90][5]       //将（帅）着法模板匹配数组
const char HorseMove[90][10]     //马的着法模板匹配数组
const char HorseLegs[90][10]      //蹩马腿模板匹配数组
const char BlackPawnMove[90][4] //黑卒着法模板匹配数组
const char RedPawnMove[90][4]   //红兵着法模板匹配数组
```

使用路向行向比特向量定义数组：

```
unsigned int HORIZON[10];   //行向比特向量数组
unsigned int VERTICAL[9];   //路向比特向量数组
//初始化路向行向信息（参考式(2.2)和式(2.3)）
HORIZON[9] = 511;
HORIZON[8] = 0;
HORIZON[7] = 130;
HORIZON[6] = 341;
HORIZON[5] = 0;
HORIZON[4] = 0;
HORIZON[3] = 341;
HORIZON[2] = 130;
HORIZON[1] = 0;
HORIZON[0] = 511 ;
VERTICAL [0] = 585;
VERTICAL [1] = 645;
VERTICAL [2] = 585;
VERTICAL [3] = 513;
```


中国知网 https://www.cnki.net

```
VERTICAL [4] = 585;
VERTICAL [5] = 513;
VERTICAL [6] = 585;
VERTICAL [7] = 645;
VERTICAL [8] = 585;
```

通过使用模板匹配和路向行向比特向量预置表，在着法生成时用查找代替计算，大大加快了着法生成的速度。

5.1.2 评估函数

在估值核心类 CEvaluation 中实现。本系统设计的评估函数主要考虑了棋子在开中局和残局中的位置值、棋子的灵活度值、子力互相联系、威胁及保护、士相防守缺陷、特殊棋型、车炮牵制将（帅）、马受阻碍等因素。定义代码片段如下：

```
class CEvaluation
{
public:
    ……
    void PreEvaluate(void);   //开中局棋子位置值
    int DefencePieces();      //士相防守缺陷的评价
    int ElephantBishopShape(void) const; //特殊棋型的评价
    int RookCannonImpact(void) const;  //车或炮牵制帅（将）或车的棋型的评价
    int PawnProtect(void) const;    //兵的保护附加值
    int Associate(void) const;      //子力相互联系的评价
    int HorseBlock(void) const;     //马受阻碍的评价
    CEvaluation();
    virtual ~CEvaluation();
    ……
protected:
    short m_BaseValue[15][90];  //存放固定子力值的数组
    short m_chessValue[90];     //存放每一个位置棋子的总价值
    int m_FlexValue[15];        //存放棋子灵活度系数的数组
   BYTE m_FlexibilityPos[90]; //存放每一位置棋子的灵活度分数
   BYTE m_GuardPos[90];    //存放每一位置被保护的信息
   short m_AttackPos[90];      //存放每一位置被威胁的信息
    ……
    };
```

函数 Evaluation 返回一个数值（即局面的评估值）供搜索引擎使用，与搜索引擎配合

中国知网 https://www.cnki.net

帮助电脑选择最佳的走法。

5.1.3 用户界面

用户界面即电脑和用户下棋的操作界面，有了它程序才能使用。一个好的用户界面应该包括一些基本的功能控件，例如开始新局，级别设置，悔棋，先后手，更换棋盘棋子的风格，加入背景音乐和落子声等等。另外，定义函数 IsGameOver 判断游戏是否结束，用下面几行实现：

```
IsGameOver() {
    BOOL RedLive = FALSE, BlackLive = FALSE;
    if (ChessmanIndex[1]!=INVALID)   BlackLive=TRUE;   //黑将
    if(ChessmanIndex[17]!=INVALID)   RedLive=TRUE;    //红帅
    if (RedLive && BlackLive)   return FALSE;
    else return TRUE;
}
```

图 5-2 是本文开发的博弈系统的用户界面，在对弈过程中，如果用户的操作不符合博弈规则，程序不会接受用户的输入。

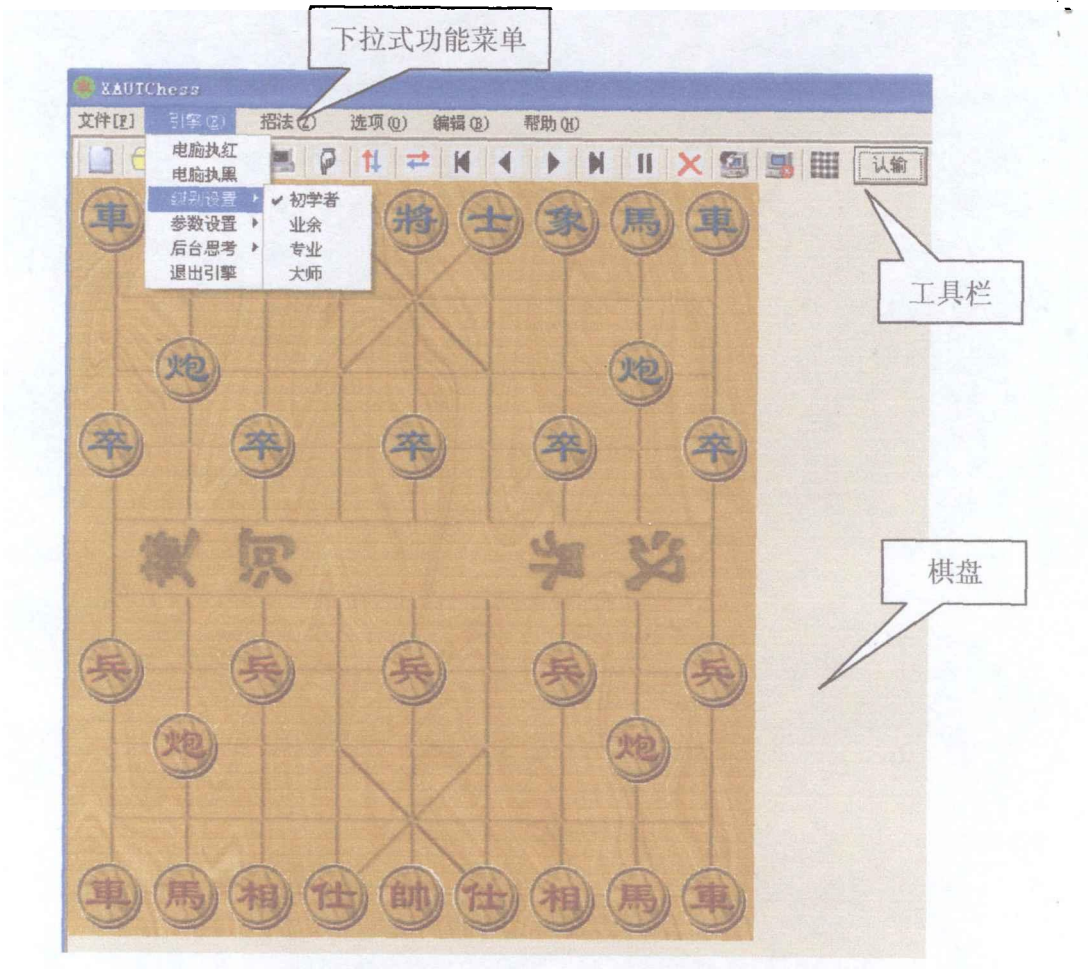


图 5-2 用户界面

Figure 5-2 The fig. of user interface

中国知网 https://www.cnki.net

5.2 测试平台

系统性能的测试是开发过程中很重要的工作。系统测试除了要检查程序需要定义的功能、排除编程中的错误、保证系统在对弈过程中顺利运行之外，更重要的是参数的优化与棋力的提高。前面提到在评估函数中就有成百上千个参数，在搜索过程中也有一系列的预置参数，都需要在调试和实际对弈过程中不断改进。

测试棋力最好的办法是通过对战平台，可以是自己对自己，也可以是与其他象棋软件自动对阵。本文开发的博弈系统的测试就是通过对战平台，让系统与人类棋手对弈或者与其他象棋软件对弈，每当程序作了较大的修改，都要通过对战检查棋力是否有所提高，是否会发生死机或出现新的问题。系统性能的测试一项复杂的工作，是非常需要耐心、又需要丰富的象棋知识的细活，要设计出高水平的博弈软件，请象棋高手参加工作是必不可少的。

另外，尽管各种程序基本方法是一样的，但是在相同的硬件条件下，它们的棋力差别可能非常大，这可能也是由于程序员对程序所作的投入，例如影响搜索速度的因素有时取决于程序使用的置换表的大小，语言代码是否简洁、规范、高效以及使用何种编译器也会导致不同的速度。

中国知网 https://www.cnki.net

6 总结及展望

中国象棋计算机博弈起步较晚，很多关键技术的研究还不够成熟，尤其对审局的研究还没有一套系统的理论，评估函数的构建与优化是整个博弈系统里最特别的一部分，参考资料少，至今还有很多盲点，或者认识不到，或者说不清好坏，这给研究和实现带来了很大困难。本文对计算机博弈的关键技术进行了全面的介绍和分析，深入研究了数据结构的设计和评估函数的构成与优化。主要工作总结如下：

(1) 分析研究了几种局面表示方法，通过比较试验，选用 0~89 一维数组编码棋盘坐标，并使用棋盘索引数组、棋子索引数组、映射数组，以及路向行向比特向量等数据结构，解决了如何把棋盘分布信息和棋盘状态信息编码进计算机的问题；采用模板匹配和路向行向比特向量预置表的数据结构加快着法生成，实验测试证明，本文设计的数据结构能有效的提高博弈系统的速度。

(2) 对评估函数建立数学模型，主要采用手工优化方法调整估值参数值，并在评估函数中加入了藐视因子和局势因子估值策略优化着法路径的选择，通过不断地对弈、调整、再对弈来优化评估函数的实际效果。对弈结果表明本文设计的评估函数是合理的，并有效提高了对弈的获胜率。

由于对该课题的研究工作刚刚开始，研究工作的时间有限，还存在很多有待解决的问题，系统性能也有很大的提升空间，在今后的工作中，应该主要从以下几个方面深入研究：

(1) 着重对评估参数优化修正的研究。对于复杂的棋类，计算机博弈研究界还没有一种很好的参数学习修正方法，将智能算法应用到计算机博弈中，主要是利用机器自学习方法自动调整评估函数的参数，而如何更好地使用这些算法优化估值参数是今后计算机博弈发展和研究的重要内容。

(2) 做好评估函数和搜索算法的结合。提高估值准确度，加强对象棋知识的理解，建立更好的评估函数数学模型，同时合理地优化程序代码，提高博弈速度是今后一段时间内发展的重点。

(3) 开局库、残局库等其他技术的设计和使用是实现高水平计算机博弈系统所必须的，也应该在以后的工作中给予重视。

目前，对中国象棋计算机博弈的研究已经吸引了越来越多的学者和研究机构投入其中，虽然很多工作还处在起步阶段，许多问题有待解决，但前景不容忽视。当前的计算机博弈水平正在迅速地提高，随着博弈理论的进一步成熟完善和计算机硬件水平的不断发展，可以相信在不久的将来，中国象棋计算机博弈将会有更广阔的天地，战胜世界冠军将不只是一个梦想。对计算机博弈的研究，也必将会在人工智能的发展中发挥更加重要的作用。

中国知网 https://www.cnki.net

致 谢

首先非常感谢我尊敬的导师钱富才教授!本论文是在钱老师的悉心指导与关怀下完成的，钱老师既是一位知识渊博，治学严谨求实，深受学生爱戴的好老师，又是一位对科研工作一丝不苟，充满热情的学者。三年来他不仅在课题的研究中给予了我很多教导，督促和鼓励，并且使我在专业的其他方面得到了很大的发展和锻炼，在课题研究遇到困难时，他对我的鼓励和谆谆教诲至今仍响在我的耳边，激励我不断向前。在硕士研究生学习即将结束之际，在此向钱老师表达我最诚挚的感谢，祝愿他身体健康，工作顺利，生活幸福!

感谢刘龙老师,在课题的研究与论文的写作过程中得到了刘老师很多细致的指导和有益的建议，使我能够有条不紊、顺利地完成课题的研究工作。感谢刘丁教授、李琦教授和郑岗教授，感谢刘涵老师、任海鹏老师、杨延西老师、赵跃老师、辛菁老师以及西安理工大学信控中心全体老师和博士的帮助,他们为课题的研究和论文的写作提供了很多便利条件，他们忘我、饱满的工作热情和不断追求进步的科研精神给了我很深的影响，在此表示感谢，谢谢他们的无私帮助和指导。

感谢谢国同学，在一起完成本课题期间，他乐观向上，积极进取的态度一直影响着我，他对课题的深刻理解和在很多方面给予我的建议，都使我受益匪浅。虽然期间有很多次因为对某些方面的理解有分歧，但他的团队合作精神以及对我的包容，忍让和鼓励，使我们的合作非常成功也非常愉快，与他建立的深厚友谊是我人生一笔宝贵的财富!

感谢张兴彦同学在课题研究期间给予的帮助，感谢吴雄君，庄元，张绚蕾，王瑞，杨珺，王立敏，赵淼，肖圳杰，叶平，曾伟，苏鑫，李皓，刘鹏飞，李文超，成斐鸣等同学，无论是在学习还是生活上,他们让我深深体会到我们这个集体是一个充满活力,温馨和睦，互帮互助的集体，有幸与他们同窗学习共同进步，是我一生珍惜的回忆。

最后感谢我的家人和朋友。父母勤劳简朴，任劳任怨且永远包容，始终是我的坚强后盾，激励我战胜挫折克服困难，不断前进。借此机会，向我远在千里之外的家人、亲朋表示我衷心的感激和谢意，祝愿他们幸福安康!

中国知网 https://www.cnki.net

参考文献

【1】 范如国，韩民春. 博弈论[M]. 武汉大学出版社，2006

【2】 李广久. 博弈论基础教程[M]. 化学工业出版社，2005

【3】 Claude E. Shannon. Programming a computer for playing chess[J], Philosophical Magazine, 1950, Vol. 41:256-275

【4】 陆汝钤. 人工智能上册[M]. 北京：科学出版社，1995

【5】 [美]Nils J. Nilsson. 人工智能[M]. 机械工业出版社，2000

【6】 电脑国际象棋简史 http://www.elephantbase.net/computer/history.htm

【7】 象棋百科全书 http://www.elephantbase.net

【8】 T.A.Marsland. COMPUTER CHESS AND SEARCH [R], Computing Science Department, University of Alberta, 1992

【9】 G. M. Adelson-Velesky, V. L. Arlazarov M. V. Donskoy. Some methods of Controlling the Tree Search in Chess Programs[J], Artificial Intelligence, Vol.6,1975, pp.361-371

【10】徐心和，王骄. 中国象棋计算机博弈关键技术分析[J]. 小型微型计算机系统，2006.6, Vol.27.No.6, pp.961-969

【11】Michael Buro. From simple features to sophisticated evaluation functions [M]. Proceedings of the First International Conference on Computers and Games (CG-98), Lecture Notes In Computer Science, 1998, Vol. 1558, pp.126-145

【12】王小春. PC游戏编程[M]. 重庆大学出版社，2002

【13】涂志坚. 电脑象棋的设计与实现[D]，中山大学，硕士论文，2004

【14】S.Cracraft. Bitmap Move Generation in Chess [J]. Journal of the International Computer Chess Association, Vol.7, No.3, pp. 146-153

【15】Huang WQ, Song EM. Never Lose Algorithm in XiangQi [M]. HuaZhong University of Sience and Technology press, 1995.5

【16】Shi-Jim Yen, Jr-Chang Chen, Tai-Ning Yang. COMPUTER CHINESE CHESS [J], ICCA, 2004.3.

【17】Aske Plaat. Research Re: Search & Re-search [D]. Ph.D. thesis, Erasmus University, Rotterdam, The Netherlands, 1996

【18】R.L. Rivest. Game Tree Searching by MinMax Approximation [J]. Artificial Intelligence, 1987, Vol.34, No.1.:77-96

【19】Qian Liang. The Evolution Of Mulan: Some Studies In Game-Tree Pruning And Evaluation Function In The Game Amazons [D]. Master of Science Thesis, University of New Mexico Albuquerque, New Mexico, 2003

【20】Knuth D.E., Moore R. W. An analysis of alpha-beta pruning [J]. Artificial Intelligence, 1975, 6(4): 293-326

中国知网 https://www.cnki.net

【21】 Schaeffer J. The History Heuristic and Alpha-Beta Search Enhancements in Practice [J]. IEEE Trans. Pattern Anal. Mach. Intell. ,1989,11(11): 1203-1212

【22】 Sashi Lazar. Analysis of Transposition Tables and Replacement Schemes [R]. Technical report, University of Maryland, 1995

【23】 Sakuta M, Hashimoto T, Nagashima J, et al. Application of the killer-tree heuristic and the lambda-search method to lines of action [J]. Information Science, 2003, 154(3-4): 141-155

【24】 Ernst A. Heinz. Adaptive null-move pruning [J]. ICCA Journal, Vol.22, No.3, pp.123-132, 1999

【25】 Christian Donninger. Null move and deep search: Selective search heuristics for obtuse chess programs [J]. ICCA Journal, 16(3):137–143, 1993

【26】 Omid David Tabibi, Nathan S. Netanyahu. Verified Null-Move Pruning [J]. ICGA Journal. 25(3): 153-161, 2002

【27】 Marsland T A. A Review of Game-Tree Pruning [J]. ICCA Journal, 1986, 9(1): 3-19

【28】 Andreas Junghanns, J. Schaeffer. Search Versus Knowledge in Game-Playing Programs Revisited [C]. Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence (IJCAI-97), 1997: 692-697

【29】 肖齐英，王正志. 博弈树搜索与静态估值函数[J]，计算机应用研究，1997

【30】 王骄，王涛，罗艳红，徐心和. 中国象棋计算机博弈系统评估函数的自适应遗传算法实现[J]. 东北大学学报（自然科学版），2005, 26（5）：949-952

【31】 胡达. C++ Builder程序设计范例——中国象棋[M]. 清华大学出版社，2002

【32】 计算机博弈 http://www.elephantbase.net/computer.htm

【33】 王凌. 智能优化算法及其应用[M]，清华大学出版社，2001

【34】 康立山，谢云，尤矢勇等. 模拟退火算法[M]，北京：科学出版社，1994: 150-151

【35】 蔡自兴，徐光佑. 人工智能及其应用[M]，北京：清华大学出版社，2003

【36】 李士勇. 模糊控制·神经网络与智能控制论[M]，哈尔滨工业大学出版社，1996

【37】 蔡自兴. 智能控制[M]. 电子工业出版社出版社, 2004

【38】 David H.Lorenz, Shaul Markovitch. Derivative Evaluation Function Learning Using Genetic Operators [J]. Computer Science Department Technion-Israel Institute of Technology

【39】 叶品星. 一种博弈树静态估值算法——△Feature状态估值[J]. 计算机工程与设计，Jul.2004, Vol.25, No.7

【40】 周玮，张赜，周静怡. 基于对弈局势的二次估值方法[J]. 系统仿真学报，2006, Vol.18, No.9

【41】 Ozdamar L. A genetic algorithm approach to a general category project scheduling problem [J]. IEEE Transactions on Systems Man and Cybernetics Part-C (S1094-6977), 1999, 29(l): 49-59

【42】 Graham Kendall, Glenn Whitwell. An Evolutionary Approach for the Tuning of a Chess Evaluation Function using Population Dynamics [C]. Proceedings of the 2001 Congress on Evolutionary Computation CEC2001

中国知网 https://www.cnki.net

【43】 Jonathan Baxter, Andrew Tridgell, Lex Weaver. KnightCap: A chess program that learns by combining TD(λ) with minimax search [R]. Technical Report, Learning Systems Group, Australian National University, 1997

【44】 Chris McConnell. Tuning Evaluation Functions for Search [C], Proceedings of the Hong Kong Computer Strategy Game Programming Workshop

【45】 Peter J. Angeline, Jordan B.Pollack. Competitive environments evolve better solution for complex tasks [C]. Proceedings of the Fifth International Conference on Genetic Algorithms. 1993, 264-270

【46】 Michael Buro. Statistical Feature Combination for the Evaluation of Game Positions [J]. Journal of Artificial Intelligence Research 3, 1995, 373-382

【47】 A.L.Samuel. Some studies in machine learning using the game of checkers [J]. IBM Journal of Research and Development. Vol.3, No.3: 210-229

【48】 Hsu S.C. and Tsao K.M. Design and Implementation of an Opening Game Knowledge-Base System for Computer Chinese Chess [J]. Bulletin of the College of Engineering, N.T.U., 1991, No. 53, pp. 75-86

【49】 A.Zobrist. A New Hashing Method with Application for Game Playing [R]. Technical Report 88, Computer Science Department, University of Wisconsin, Madison. 1970

【50】 Ren Wu, Donald F. Beal. A Memory Efficient Retrograde Algorithm and Its Application to Chinese Chess Endgames [J]. More Games of No Chance, MSRI Publications, 2002, Volume 42, pp.213-227

【51】 Chen S H. Design and Implementation of a Practical Endgame Database for Chinese Chess [C]. M.Sc.Thesis, Department of Computer Science and Information Engineering, National Taiwan University, Taiwan

【52】 棋天大圣网站 http://www.neuchess.com/

【53】 机器博弈学术研讨会网站 http://www.aigames.cn/

中国知网 https://www.cnki.net